Frankestein Transformer: Biblioteca Unificada Codificador-Decodificador, CLI y Notas de Diseño Basadas en Investigación

Erick F. Merino M.
Este trabajo no está afiliado
erickfmm@gmail.com

Febrero 2026

Resumen

Frankestein Transformer presenta un kit de herramientas unificado basado en configuración para la experimentación sistemática con arquitecturas de transformadores modernas, abarcando diecisiete variantes de mezclador de secuencias y veinte familias de optimizadores. El sistema respalda tanto el entrenamiento estilo codificador con modelado de lenguaje enmascarado (MLM) como el entrenamiento estilo decodificador con predicción autorregresiva de siguiente token, con flujos de trabajo de ajuste fino especializados para ambas arquitecturas. Las contribuciones de investigación son triples: (i) un contrato de configuración basado en esquema estricto que permite la experimentación reproducible en diversos mecanismos de atención, incluyendo atención softmax estándar, atención sigmoide, redes retentivas, modelos de espacio de estado selectivos, transformadores de profundidad continua, atención aumentada con memoria, patrones de atención dispersa y mecanismos con compuerta; (ii) un marco integral de enrutamiento de optimizadores que admite métodos de reducción de varianza (MARS, Adan, AdEMAMix), variantes eficientes en memoria (Adafactor, GaLore, Lion), enfoques sin programador y precondicionadores de segundo orden (Shampoo, SOAP, Sophia); y (iii) flujos de trabajo de extremo a extremo que abarcan el despliegue cuantificado mediante empaquetado de pesos ternarios y entrenamiento de incrustaciones de oraciones inspirado en SBERT. El kit de herramientas implementa una interfaz de configuración basada en web que proporciona renderizado de formularios impulsado por esquemas con documentación en línea y validación en tiempo real. Este documento de referencia técnica incluye diagramas arquitectónicos, visualizaciones de flujo de ejecución, tablas de decisión y apéndices integrales que sintetizan la literatura sobre arquitecturas de transformadores, mecanismos de atención dispersa, variantes de atención con compuerta y algoritmos de optimización. El sistema permite iteración rápida mientras mantiene condiciones experimentales reproducibles a través de su filosofía de diseño basada primero en esquemas.

Índice

1. Introducción **7**
1.1. Motivación y Planteamiento del Problema . . . 7
1.2. Contribuciones . . . 8
1.3. Guía de Lectura . . . 8
1.4. Constructor de Configuración Basado en Web . . . 9
1.5. Novedades Recientes: Soporte de Decodificador y Reorganización de Configuración 10
1.5.1. Implementación de Decodificador Autoregresivo . . . 10
1.5.2. Reorganización de Configuración para Accesibilidad Mejorada . . . 10

2. Introducción Conceptual: Transformers y Mecanismo de Atención para Novatos **11**
2.1. ¿Qué es un Transformer? . . . 11
2.2. El Mecanismo de Atención: Una Analogía . . . 11
2.3. Ejemplo Práctico Paso a Paso . . . 11
2.3.1. Contexto 1: "El gato come pescado" . . . 12
2.3.2. Contexto 2: "El restaurante come clientes a buen precio" . . . 12
2.4. Cómo Funciona la Atención Matemáticamente (Versión Simple) . . . 12
2.4.1. Paso 1: Preguntas, Claves y Valores . . . 12
2.4.2. Paso 2: Compatibilidad . . . 12
2.4.3. Paso 3: Enfoque . . . 12
2.4.4. Paso 4: Combinar . . . 13
2.5. Múltiples Cabezas de Atención: Exploración Múltiple . . . 13
2.6. Apilamiento de Capas . . . 13
2.7. ¿Por Qué es Importante? . . . 13
2.8. Desafíos Prácticos . . . 13
2.8.1. Complejidad Computacional . . . 14
2.8.2. Requisitos de Memoria . . . 14
2.8.3. Eficiencia en Dispositivos . . . 14
2.9. Soluciones Modernas . . . 14
2.10. Resumen para Llevar . . . 14

3. Trabajo Relacionado **15**
3.1. Arquitecturas de Mezclador de Secuencias . . . 15
3.1.1. Líneas Base de Atención Densa . . . 15
3.1.2. Arquitecturas Recurrentes y Retentivas . . . 16
3.1.3. Mecanismos de Atención Dispersa . . . 16
3.1.4. Mecanismos de Memoria con Compuerta . . . 17
3.2. Algoritmos de Optimización Avanzados . . . 18

4. Arquitectura Centrada en Configuración **18**
4.1. Contrato de Esquema y Validación . . . 18
4.2. Validación de Esquema . . . 19
4.3. Selección de Clase de Modelo . . . 19
4.4. Selección de Modo de Entrenamiento . . . 19
4.5. Tipos de Tarea de Entrenamiento . . . 20

5. Taxonomía de Arquitectura e Implementación **20**
5.1. Familias de Atención y Mezclador de Secuencias . . . 20
5.2. Atención Estándar . . . 21
5.3. Atención Sigmoide . . . 21
5.4. Formulación Retentiva . . . 22
5.5. SSM Selectivo (Mamba) . . . 22
5.6. Actualizaciones Continuas Estilo ODE . . . 22
5.7. Memoria en Tiempo de Prueba (Titans) . . . 22
5.8. Extensiones Dispersas y con Compuerta en la Base de Código Actual . . . 22

6. Familias de Optimizadores y Dinámica de Entrenamiento **22**
6.1. Organización Taxonómica de Optimizadores Soportados . . . 23
6.2. Formulación Adaptativa Central . . . 23

7. Cuantización y Despliegue **23**
7.1. Cuantización Ternaria . . . 24
7.2. Cuantización de Activaciones . . . 24
7.3. Estimaciones de Tamaño . . . 24

8. Tareas Posteriores SBERT **24**

9. Tablas Resumen **25**
9.1. Resumen de Atención y Mezclador de Secuencias . . . 25
9.2. Resumen de Optimizadores . . . 25

10. Discusión **26**
10.1. Compromisos de Diseño Impulsado por Esquemas . . . 26
10.2. Cobertura Arquitectónica y Brechas . . . 26
10.3. Fragmentación del Paisaje de Optimizadores . . . 27
10.4. Consideraciones de Despliegue y Producción . . . 27
10.5. Desafíos de Integración y Extensibilidad . . . 27

11. Conclusión **27**

A. Arquitecturas Fundamentales de Transformadores: Análisis Teórico y Empírico Integral **28**
A.1. Atención Estándar: La Fundamentación de la Contextualización Global . . . 29
A.1.1. Formulación Matemática y Mecanismo de Enrutamiento de Tokens . . . 29
A.1.2. Complejidad Computacional y el Cuello de Botella del Caché KV . . . 29
A.1.3. Perfil Arquitectónico: Atención Estándar . . . 30
A.2. Atención Sigmoide: Localidad Algorítmica Consciente del Hardware . . . 30
A.2.1. Fundamentación Matemática y Perspectiva de Mezcla de Expertos . . . 30
A.2.2. Estabilización y el Requisito de Norma Híbrida . . . 31
A.2.3. Complejidad Computacional y Optimización de Hardware . . . 31
A.2.4. Perfil Arquitectónico: Atención Sigmoide . . . 32
A.3. RetNet: La Dualidad de Recurrencia y Atención . . . 32
A.3.1. Fundamentación Matemática y el Mecanismo de Retención Multi-Escala . . . 32
A.3.2. Complejidad Computacional y Escalado de Rendimiento . . . 33
A.3.3. Perfil Arquitectónico: RetNet . . . 33
A.4. Mamba: Modelado Selectivo de Espacio de Estado . . . 34
A.4.1. Fundamentación Matemática y el Mecanismo de Selección . . . 34
A.4.2. Complejidad Computacional y el Escaneo Consciente del Hardware . . . 35
A.4.3. Perfil Arquitectónico: Mamba . . . 35
A.5. Transformador ODE: Generación de Secuencias vía Sistemas Dinámicos . . . 36
A.5.1. Fundamentación Matemática y Refinamiento Runge-Kutta . . . 36
A.5.2. Complejidad Computacional y el Compromiso de Precisión . . . 37
A.5.3. Perfil Arquitectónico: Transformador ODE . . . 37
A.6. Titans: Memorización Neural en Tiempo de Prueba . . . 38
A.6.1. Fundamentación Matemática y la Métrica de Sorpresa . . . 38
A.6.2. Complejidad Computacional e Contexto Infinito . . . 39
A.6.3. Perfil Arquitectónico: Titans . . . 39
A.7. Síntesis Comparativa: Posicionamiento Arquitectónico . . . 39
A.7.1. Guía de Selección . . . 40

A. Tipos de Atención Dispersa en Transformadores: Revisión Exhaustiva **40**
A.1. Resumen Ejecutivo . . . 40
A.2. Sparse Transformer (Child et al., 2019) . . . 40
A.2.1. Descripción . . . 40
A.2.2. Formulación Matemática . . . 41
A.2.3. Ventajas y Desventajas . . . 41
A.3. Longformer (Beltagy et al., 2020) . . . 41
A.3.1. Descripción . . . 41
A.3.2. Formulación Matemática . . . 41
A.3.3. Ventajas y Desventajas . . . 42
A.4. BigBird (Zaheer et al., 2020) . . . 42
A.4.1. Descripción . . . 42
A.4.2. Formulación Matemática . . . 42
A.4.3. Ventajas y Desventajas . . . 43
A.5. SparseK Attention (Lou et al., 2024) . . . 43
A.5.1. Descripción . . . 43
A.5.2. Formulación Matemática . . . 43
A.5.3. Ventajas y Desventajas . . . 44
A.6. NSA — Native Sparse Attention (Yuan et al., 2025) . . . 44
A.6.1. Descripción . . . 44
A.6.2. Formulación Matemática . . . 44
A.6.3. Ventajas y Desventajas . . . 45
A.7. SpargeAttn (Zhang et al., 2025) . . . 45
A.7.1. Descripción . . . 45
A.7.2. Formulación Matemática . . . 45
A.7.3. Ventajas y Desventajas . . . 46
A.8. FASA — Frequency-Aware Sparse Attention (Wang et al., 2026) . . . 46
A.8.1. Descripción . . . 46
A.8.2. Formulación Matemática . . . 46
A.8.3. Ventajas y Desventajas . . . 47
A.9. Tabla Comparativa Resumen . . . 47
A.10. Guía de Selección y Recomendaciones . . . 47
A.10.1. Para Máximo Desempeño . . . 47
A.10.2. Para Inferencia Eficiente . . . 47
A.10.3. Para Contextos Extremadamente Largos . . . 48
A.10.4. Para Dispositivos Limitados en Memoria . . . 48
A.10.5. Para Modelos Existentes Sin Reentrenamiento . . . 48

A. Bloques de Atención con Compuerta en Transformadores: Una Revisión Completa **48**
A.1. Introducción y Motivación . . . 48
A.2. 1. Atención Lineal con Compuerta (GLA) . . . 48
A.2.1. Descripción . . . 48
A.2.2. Formulación Matemática . . . 48
A.2.3. Pseudocódigo . . . 49
A.2.4. Ventajas y Limitaciones . . . 49
A.3. 2. DeltaNet . . . 49
A.3.1. Descripción . . . 49
A.3.2. Formulación Matemática . . . 50
A.3.3. Pseudocódigo . . . 50
A.3.4. Ventajas y Limitaciones . . . 50

A.4. 3. Gated DeltaNet . . . 50
A.4.1. Descripción . . . 50
A.4.2. Formulación Matemática . . . 51
A.4.3. Pseudocódigo . . . 51
A.4.4. Ventajas y Limitaciones . . . 51
A.5. 4. RetNet (Red Retentiva) . . . 52
A.5.1. Descripción . . . 52
A.5.2. Formulación Matemática . . . 52
A.5.3. Pseudocódigo . . . 52
A.5.4. Ventajas y Limitaciones . . . 52
A.6. 5. HGRN2 (Red Recurrente con Compuertas Jerárquicas 2) . . . 52
A.6.1. Descripción . . . 52
A.6.2. Formulación Matemática . . . 53
A.6.3. Pseudocódigo . . . 53
A.6.4. Ventajas y Limitaciones . . . 54
A.7. 6. Transformador de Olvido (FoX) . . . 54
A.7.1. Descripción . . . 54
A.7.2. Formulación Matemática . . . 54
A.7.3. Pseudocódigo . . . 54
A.7.4. Ventajas y Limitaciones . . . 54
A.8. 7. Atención con Compuerta (Sigmoide después de SDPA) . . . 54
A.8.1. Descripción . . . 54
A.8.2. Formulación Matemática . . . 55
A.8.3. Pseudocódigo . . . 56
A.8.4. Ventajas y Limitaciones . . . 56
A.9. Tabla Comparativa Resumen . . . 56
A.10. Taxonomía y Evolución de Arquitecturas de Compuerta . . . 56
A.10.1. Generación 1 — Compuertas Fijas/Simples . . . 56
A.10.2. Generación 2 — Compuertas Dependientes de Datos . . . 56
A.10.3. Generación 3 — Regla Delta + Compuertas . . . 57

A. Algoritmos Avanzados de Optimización en Arquitecturas Transformadores 57
A.1. Introducción: Evolución de la Optimización en Redes Neuronales . . . 57
A.2. Optimizadores de Línea Base y Adaptativos . . . 57
A.2.1. SGD con Momentum . . . 57
A.2.2. Adam y AdamW . . . 58
A.2.3. RAdam (Rectificado Adam) . . . 59
A.3. Momentum Avanzado y Reducción de Varianza (2024-2025) . . . 59
A.3.1. Adan (Momentum Nesterov Adaptivo) . . . 59
A.3.2. ADOPT (Adam Modificado con Convergencia Óptima) . . . 59
A.3.3. AdEMAMix (Mezcla de Dos EMAs) . . . 60
A.3.4. MARS (Make Variance Reduction Shine) . . . 60
A.3.5. Optimizadores Cautelosos (C-AdamW, C-Lion) . . . 61
A.4. Optimizadores Eficientes en Memoria y a Gran Escala . . . 62
A.4.1. LAMB (Aprendizaje Adaptativo Basado en Capas) . . . 62
A.4.2. Schedule-Free (AdamW sin Cronograma) . . . 62
A.4.3. Adafactor (Factorización Inteligente de Varianza) . . . 62
A.4.4. GaLore (Proyección de Gradientes de Bajo Rango) . . . 63
A.5. Métodos Libres de Parámetros y Adaptativos a Distancia . . . 63
A.5.1. Prodigy (Adaptación de Distancia Dinámica) . . . 63
A.6. Optimizadores de Segundo Orden y Geométricos . . . 63

A.6.1. Shampoo (Precondicionadores Factorizados Kronecker) 63
A.6.2. SOAP (Shampoo con Adam en Base Propia) 64
A.6.3. Sophia (Hessiana Diagonal y Clipping) 64
A.7. Optimizadores Basados en Ortogonalidad 64
A.7.1. Lion (EvoLved Sign Momentum) 64
A.7.2. Muon (Ortonormalización Newton-Schulz) 65
A.7.3. Turbo-Muon (Muon Acelerado con Preacondicionamiento Casi-Ortogonal) 65
A.8. Tabla Comparativa: Síntesis de Optimizadores 66
A.9. Conclusión del Apéndice de Optimización 66

Bibliografía **66**

1. Introducción

1.1. Motivación y Planteamiento del Problema

Las arquitecturas de transformadores han transformado fundamentalmente el panorama del modelado de secuencias en el procesamiento del lenguaje natural [35], visión por computadora y biología computacional. El éxito de modelos como BERT [9], GPT y sus variantes ha establecido al transformador como el estándar de facto para el aprendizaje de representaciones en aprendizaje profundo. Sin embargo, la rápida proliferación de innovaciones arquitectónicas presenta desafíos prácticos significativos para investigadores y profesionales.

Los años recientes han presenciado una explosión de mecanismos alternativos de atención y mezcla de secuencias, cada uno abordando limitaciones específicas de la atención softmax estándar: complejidad computacional cuadrática [7], requisitos de inferencia eficiente en memoria [33, 10], gestión de estado selectivo basada en contenido [2] y optimizaciones conscientes del hardware [27]. Simultáneamente, la literatura de optimización se ha diversificado más allá del AdamW clásico, introduciendo técnicas de reducción de varianza [39, 24, 45], variantes eficientes en memoria [30, 49], enfoques sin programador [8] y métodos de precondicionamiento de segundo orden [12, 36].

La consecuencia práctica es un ecosistema de investigación fragmentado donde la comparación experimental a través de arquitecturas y optimizadores requiere un esfuerzo de ingeniería significativo. Los investigadores deben implementar y depurar múltiples variantes desde cero, garantizar canalizaciones de entrenamiento consistentes y gestionar espacios de hiperparámetros complejos. Esta fragmentación obstaculiza la reproducibilidad, ralentiza el progreso científico y aumenta la barrera de entrada para nuevos investigadores.

Este trabajo aborda estos desafíos a través de un kit de herramientas de experimentación unificado basado en configuración disponible en `https://github.com/erickfmm/frankestein-transformer` que proporciona:

1. **Diseño Basado Primero en Esquemas**: Un contrato de configuración estricto y validado que impone reproducibilidad mientras admite diecisiete arquitecturas distintas de mezclador de secuencias y veinte familias de optimizadores.

2. **Entrenamiento Agnóstico a la Arquitectura**: Infraestructura de entrenamiento común que admite líneas base de atención densa (estándar, sigmoide), alternativas recurrentes (RetNet, Mamba, bloques estilo ODE), patrones de atención dispersa (Sparse Transformer, Longformer, BigBird, SparseK, NSA, SpargeAttn, FASA) y mecanismos con compuerta (GLA, DeltaNet, Gated DeltaNet, HGRN2, FoX, Gated Softmax).

3. **Marco de Enrutamiento de Optimizadores**: Grupos de hiperparámetros con prefijo que permiten control granular sobre incrustaciones, capas de normalización, bloques recurrentes, bloques de atención y otros subconjuntos de parámetros a través de optimizadores diversos.

4. **Flujos de Trabajo de Extremo a Extremo**: Despliegue integrado mediante cuantización (empaquetado de pesos ternarios, activaciones INT8) y capacidades de incrustación de oraciones inspirado en SBERT [28].

5. **Configuración Interactiva**: Interfaz basada en web que proporciona renderizado de formularios impulsado por esquemas, validación en tiempo real y generación de comandos CLI.

La superficie de comandos del proyecto es:

```
frankestein-transformer
```

con subcomandos: `train`, `deploy`, `quantize`, `infer`, `sbert-train`, `sbert-infer`, `web-server`.

El comando `web-server` lanza un constructor de configuración basado en Streamlit que proporciona:

- Campos de formulario impulsados por esquemas con títulos de parámetros y descripciones detalladas
- Información sobre herramientas y texto de ayuda en tiempo real para cada opción de configuración
- Vista previa y descarga en tiempo real de YAML
- Comandos CLI generados para entrenamiento, despliegue, inferencia y flujos de trabajo SBERT

Esta interfaz interactiva sirve como alternativa a la edición manual de YAML, mejorando la usabilidad para usuarios que exploran opciones de configuración disponibles y comprenden su impacto en el comportamiento del modelo y la dinámica de entrenamiento.

1.2. Contribuciones

Este trabajo realiza las siguientes contribuciones principales:

1. **Esquema de Configuración Unificado**: Un contrato de esquema basado en YAML con validación estricta que admite diecisiete arquitecturas distintas de mezclador de secuencias a través de cuatro categorías (líneas base densas, bloques recurrentes/retentivos, patrones de atención dispersa y mecanismos con compuerta) mientras impone reproducibilidad a través de restricciones `additionalProperties: false`.
2. **Soporte de Arquitectura Integral**: Implementación de variantes modernas de transformador incluyendo atención softmax estándar [35], atención sigmoide [27], RetNet [33], Mamba [10], transformadores continuos estilo ODE [47], atención aumentada con memoria Titans [2], mecanismos de atención dispersa [7, 3, 46, 21, 44, 48, 38] y arquitecturas de atención con compuerta [41, 42, 43, 25, 17, 26].
3. **Marco de Enrutamiento de Optimizadores**: Sistema de hiperparámetros con prefijo que permite control por grupo de parámetros a través de veinte optimizadores, incluyendo métodos de reducción de varianza (MARS, Adan, AdEMAMix, Cautious AdamW), variantes eficientes en memoria (Adafactor, GaLore, Lion), enfoques sin programador (Schedule-Free AdamW), métodos conscientes de la curvatura (Sophia), precondicionadores de segundo orden (Shampoo, SOAP) y optimizadores orientados a la ortogonalidad (Muon, Turbo-Muon).
4. **Cuantización y Despliegue**: Canalización de despliegue integrada que admite empaquetado de pesos ternarios y cuantización de activaciones INT8 con estimaciones de tamaño siguiendo la aproximación de almacenamiento de 1,58 bits.
5. **Flujos de Trabajo de Incrustación de Oraciones**: Canalizaciones de entrenamiento e inferencia inspiradas en SBERT que admiten puntuación de similitud, recuperación, agrupamiento y exportación de incrustaciones persistentes.
6. **Interfaz de Configuración Interactiva**: Servidor web basado en Streamlit que proporciona generación de formularios impulsada por esquemas, validación en tiempo real, documentación en línea y generación de comandos CLI.

1.3. Guía de Lectura

Este documento está organizado como referencia técnica que aborda cuatro preocupaciones operacionales:

1. **Contrato de Configuración**: La sección 4.1 describe el esquema YAML que impone experimentos válidos y la sección 4.2 explica las reglas de validación.

2. **Selección de Arquitectura**: La sección **??** cubre opciones de normalización; la sección 5.1 proporciona comparación integral de familias de mezcladores de secuencias; los apéndices **??**, **??** y **??** sintetizan literatura de apoyo.

3. **Dinámica de Optimización**: La sección 6 detalla el enrutamiento de optimizadores y la dinámica de entrenamiento; el apéndice **??** proporciona análisis integral de familias de optimizadores.

4. **Despliegue e Inferencia**: Las secciones 7 y 8 describen el despliegue cuantizado y los flujos de trabajo de incrustación de oraciones.

1.4. Constructor de Configuración Basado en Web

Además de la edición directa de YAML, este proyecto proporciona una interfaz web basada en Streamlit (accedida a través del comando `web-server`) que mejora la accesibilidad y descubrimiento de configuración. La interfaz presenta campos de esquemas con:

- **Renderizado de formularios impulsado por esquemas** — Todos los campos se generan dinámicamente desde el esquema autoritativo, garantizando consistencia y validación.
- **Documentación de parámetros en línea** — Cada campo de formulario muestra un *título* del esquema como su etiqueta, con la *descripción* mostrada como ayuda al pasar el cursor.
- **Vista previa de configuración en tiempo real** — Los usuarios ven salida YAML en vivo mientras modifican campos de formulario, permitiendo retroalimentación de validación inmediata.
- **Generación de comandos CLI** — La interfaz genera comandos CLI completos para entrenamiento, despliegue, inferencia y flujos de trabajo SBERT basados en la configuración actual.
- **Mejoras de accesibilidad** — Información sobre herramientas y formularios estructurados hacen que las opciones de configuración sean más fáciles de entender, especialmente para usuarios nuevos en el proyecto o explorando arquitecturas novedosas.

Este enfoque basado en web aborda barreras de usabilidad comunes en la experimentación impulsada por configuración:

- Reduce la necesidad de memorizar la estructura de YAML y los nombres de campos
- Previene errores tipográficos a través de validación de esquema
- Proporciona contexto educativo a través de documentación en línea
- Permite experimentación rápida con ajuste guiado de parámetros
- Sirve tanto como herramienta de configuración como recurso de aprendizaje

La implementación de la interfaz web utiliza widgets de formulario de Streamlit con:

- `st.checkbox()` para interruptores binarios con texto de ayuda
- `st.number_input()` para campos numéricos con tamaño de paso y formato
- `st.selectbox()` para opciones de enumeración con visualización de opciones
- `st.multiselect()` para selecciones de matriz desde opciones definidas

- `st.text_input()` para campos de texto
- `st.info()`, `st.caption()` para información complementaria

Los metadatos del esquema (campos de título y descripción) se extraen y renderizan sistemáticamente a través de todas las secciones de formulario, incluyendo:

- Parámetros de arquitectura del modelo (tamaño oculto, capas, cabezas de atención, etc.)
- Configuraciones de tiempo de ejecución de entrenamiento (tamaño de lote, acumulación, programador)
- Configuración de optimizador con hiperparámetros por grupo de parámetros
- Opciones de despliegue y cuantización
- Parámetros de entrenamiento e inferencia específicos de SBERT

1.5. Novedades Recientes: Soporte de Decodificador y Reorganización de Configuración

Esta versión introduce dos mejoras significativas de arquitectura:

1.5.1. Implementación de Decodificador Autoregresivo

El sistema ahora soporta tanto arquitecturas estilo codificador como decodificador a través de la nueva clase `model_class='frankesteindecoder'`, permitiendo experimentos unificados de modelado de lenguaje bidireccional (MLM estilo BERT) y generación autorregresiva (AR estilo GPT) dentro de la misma canalización de entrenamiento. El soporte de decodificador incluye:

- **Enmascaramiento Causal**: El campo `model.mode='decoder'` activa enmascaramiento causal automático en todas las capas de atención, permitiendo solo que cada token atienda a tokens anteriores en la secuencia.
- **Predicción de Siguiente Token**: Objetivo de entrenamiento autorregresivo que predice el siguiente token de la secuencia basado solo en contexto causal, replicando la dinámica de entrenamiento de modelos estilo LLM.
- **Flujos de Trabajo de Generación**: Herramientas integradas para inferencia autorregresiva, incluyendo sampleo con temperatura, top-k, top-p y construcción de secuencias de longitud variable con parada por token EOS.
- **Compatibilidad de Mezcladores**: Todas las diecisiete variantes de mezclador de secuencias (atención densa, recurrente, dispersa y con compuerta) se integran automáticamente en modo decodificador con enmascaramiento causal aplicado transparentemente.

1.5.2. Reorganización de Configuración para Accesibilidad Mejorada

Los archivos de esquema de configuración ahora están disponibles directamente desde el directorio raíz del proyecto, además de su ubicación tradicional en `configs/`, mejorando la discoverabilidad y simplificando la referencia en flujos de trabajo de CI/CD e integración de desarrollo. Beneficios incluyen:

- **Accesibilidad Mejorada**: El archivo `schema.yaml` se puede referenciar desde la raíz del proyecto sin navegar subdirectorios, reduciendo fricción para usuarios y herramientas.

- **Carga de Configuración Simplificada**: Las rutas relativas en scripts de automatización se simplifican cuando los esquemas están a nivel de raíz, habilitando canalizaciones de reproducibilidad más limpias.
- **Generación por Herramientas**: Las herramientas de generación de documentación y validadores externos pueden ubicar el contrato de esquema sin conocimiento específico de la estructura de directorios interna del proyecto.
- **Compatibilidad Hacia Atrás**: Los archivos en `configs/` se mantienen para compatibilidad hacia atrás; el sistema prioriza automáticamente correctamente según disponibilidad según la ruta de búsqueda.

2. Introducción Conceptual: Transformers y Mecanismo de Atención para Novatos

Esta sección explica los conceptos fundamentales de los transformers y el mecanismo de atención de manera accesible, sin requerir conocimiento previo profundo en aprendizaje profundo. Los transformers son el motor detrás de sistemas de inteligencia artificial modernos como ChatGPT, pero su funcionamiento puede entenderse con analogías del mundo real.

2.1. ¿Qué es un Transformer?

Un transformer es una arquitectura neural que procesa textos o secuencias de datos interpretando el significado de cada palabra considerando todas las demás palabras en el contexto. Imagina que lees una oración:

"El banco estaba lleno de gente esperando. María fue al banco."

¿Qué significa "banco" en cada caso? En la primera oración, se refiere a una institución financiera (o un asiento). En la segunda, es menos claro sin contexto. Un transformador resuelve esto automáticamente mirando todas las palabras circundantes. En la segunda oración, al ver "María fue", el modelo entiende mejor que probablemente sea un banco de peces (donde va a nadar) o una institución (donde va a hacer un trámite).

Esta capacidad de entender el significado completo de una palabra considerando toda la oración se llama *contextualización*, y es el corazón de cómo funcionan los transformers modernos.

2.2. El Mecanismo de Atención: Una Analogía

El mecanismo de atención es el "truco mágico" que permite a los transformers entender contexto. Piénsalo como en una conversación importante en una habitación ruidosa:

1. **Algunos esúndalo**: Alguien habla siendo muy importante para ti.
2. **Ignoras lo demás**: Tu cerebro automáticamente enfoca atención en esa persona, ignorando otros ruidos.
3. **Entiendes mejor**: Al concentrarte, captas cada palabra claramente.

El mecanismo de atención neural funciona igual: cada palabra "pregunta" qué tan importante es cada otra palabra para entender su significado, luego "enfoca" su atención en las más relevantes.

2.3. Ejemplo Práctico Paso a Paso

Veamos cómo un transformer entiende la palabra "come" en dos contextos diferentes:

2.3.1. Contexto 1: "El gato come pescado"

El transformer, al procesar "come", pregunta internamente:

- ¿Cuán relevante es "El"? Poco (es solo un artículo). **Atención baja**.
- ¿Cuán relevante es "gato"? Muy relevante (es el sujeto, quién come). **Atención alta**.
- ¿Cuán relevante es "pescado"? Muy relevante (es lo que se come). **Atención alta**.

Así, la representación mental de "come" se enfoca principalmente en "gato" y "pescado".

2.3.2. Contexto 2: "El restaurante come clientes a buen precio"

Aquí el transformer pregunta en contexto diferente:

- ¿Cuán relevante es "restaurante"? Muy relevante (es el sujeto). **Atención alta**.
- ¿Cuán relevante es "clientes"? Muy relevante (es el objeto). **Atención alta**.
- ¿Cuán relevante es "buen precio"? Relevante (es la razón). **Atención media**.

La palabra "come" obtiene una representación completamente diferente porque "atiende" a palabras distintas en contextos distintos.

2.4. Cómo Funciona la Atención Matemáticamente (Versión Simple)

Detrás del cambio de atención hay operaciones matemáticas. Aquí está la versión simplificada sin demasiado detalle técnico:

2.4.1. Paso 1: Preguntas, Claves y Valores

Cada palabra en la oración se transforma en tres versiones:

- **Pregunta (Query)**: "¿Qué información necesito conocer?"
- **Clave (Key)**: "¿Qué información tengo?"
- **Valor (Value)**: "Aquí está mi información importante."

Imagina que en una biblioteca, cada visitante es una "pregunta", cada libro es una "clave", y el contenido es el "valor". El bibliotecario (atención) empareja preguntas con las claves más relevantes para acceder al valor correcto.

2.4.2. Paso 2: Compatibilidad

El sistema examina qué tan *compatible* es cada pregunta con cada clave. Preguntas similares a las claves obtienen una puntuación de compatibilidad *alta*. Esto se calcula multiplicando la pregunta por la clave (matemáticamente, el producto punto).

2.4.3. Paso 3: Enfoque

Las puntuaciones de compatibilidad se convierten en "focus weights" (pesos de enfoque). Las compatibilidades altas significan "enfoca atención aquí", y las bajas significan "ignora esto". Matemáticamente, se usa una función llamada `softmax` que convierte puntuaciones en porcentajes (como si fuera: 40 % atención aquí, 35 % aquí, 25 % allá).

2.4.4. Paso 4: Combinar

Finalmente, el sistema combina todos los valores, ponderados por los pesos de enfoque. Si una palabra tiene 80 % de atención en el sujeto, el 80 % de la información del sujeto se mezcla en la representación de esa palabra.

2.5. Múltiples Cabezas de Atención: Exploración Múltiple

Una característica clave es que los transformers no usan una sola atención, sino *múltiples cabezas de atención* en paralelo. Es como si tuvieras 8 personas analizando la oración *simultáneamente*, cada una prestando atención a diferentes aspectos:

- **Persona 1**: "Me enfoco en relaciones de sujeto-verbo."
- **Persona 2**: "Yo busco quién es el objeto directo."
- **Persona 3**: "Yo rastreo información sobre tiempo verbal."
- **Persona 4**: "Yo busco modificadores y adjetivos."

Cada "cabeza" aprende a enfocarse en diferentes patrones de idioma. Juntas, capturan una comprensión mucho más rica que una sola cabeza.

2.6. Apilamiento de Capas

Los transformers procesan información en *múltiples capas* (usualmente 12 a 48 para modelos prácticos). Imagina corregir un ensayo:

1. **Primera pasada**: Corriges la ortografía y gramática básica.
2. **Segunda pasada**: Mejoras la claridad y la estructura de oraciones.
3. **Tercera pasada**: Aseguras consistencia y flujo narrativo.

Las transformers funcionan igual: cada capa refina la comprensión del texto. La primera capa captura características básicas (palabras simples, géneros), mientras que capas posteriores entienden conceptos más complejos (relaciones entre palabras, significados abstractos).

2.7. ¿Por Qué es Importante?

El mecanismo de atención fue revolucionario porque:

1. **Paralelismo**: Encuentra contexto de *cualquier palabra con cualquier otra palabra*, sin procesarlas secuencialmente (a diferencia de sistemas anteriores). Esto lo hace muy rápido.
2. **Flexibilidad**: Aprende qué patrones buscar automáticamente del datos, sin que necesites programar reglas manualmente.
3. **Escalabilidad**: Funciona desde textos pequeños hast contextos de millones de palabras.
4. **Capacidades Generales**: El mismo mecanismo funciona para traducción, resumen, preguntas-respuestas, generación de texto, visión por computadora, y más.

2.8. Desafíos Prácticos

A pesar del éxito extraordinario de los transformers, presentan desafíos:

2.8.1. Complejidad Computacional

Si una oración tiene 1000 palabras, el mecanismo de atención estándar debe comparar cada palabra con todas las otras 1000 palabras. Esto es $1000 \times 1000 = 1,000,000$ comparaciones. Para documentos largos con millones de palabras, esto se vuelve prohibitivamente costoso en tiempo y memoria.

2.8.2. Requisitos de Memoria

Especialmente en inferencia (cuando usas modelos entrenados), almacenar toda la matriz de atención puede consumir enormes cantidades de RAM o GPU, limitando la longitud de secuencias que puedes procesar.

2.8.3. Eficiencia en Dispositivos

Los transformers fueron diseñados para TPUs y GPUs poderosas. Ejecutarlos en teléfonos o dispositivos embebidos es desafiante.

2.9. Soluciones Modernas

Para resolver estos desafíos, la investigación ha propuesto muchas variantes:

- **Atención Dispersa**: En lugar de comparar cada palabra con TODAS las otras, solo compara con un subconjunto estratégico (vecinos cercanos, patrones periódicos). Reduce complejidad de $\mathcal{O}(n^2)$ a $\mathcal{O}(n \log n)$ o $\mathcal{O}(n)$.
- **Modelos Recurrentes**: Como Mamba, incorporan aspectos de redes neuronales recurrentes antiguas pero con mejor eficiencia moderna.
- **Atención con Compuerta**: Mecanismos que aprendan selectivamente qué información pasar forward, reduciendo necesidad de almacenamiento.
- **Cuantización**: Usar números más pequeños (enteros en lugar de decimales) para reducir memoria sin perder demasiada precisión.

Estas innovaciones son lo que esta herramienta (Frankenstein) te permite experimentar fácilmente.

2.10. Resumen para Llevar

- Un **transformer** es una red neural que entiende contexto de lenguaje muy bien.
- El **mecanismo de atención** permite a cada palabra "enfocarse" en qué otras palabras son relevantes para entender su significado.
- La atención funciona calculando **compatibilidad** entre cada palabra (pregunta) y todas las otras (claves), luego combinando información basada en esa compatibilidad (valores).
- **Múltiples cabezas** de atención exploran diferentes patrones en paralelo.
- **Múltiples capas** refinan progresivamente la comprensión.
- El desafío principal es que la atención estándar tiene complejidad cuadrática, lo cual es caro para textos largos.
- Muchas **soluciones modernas** (atención dispersa, modelos recurrentes, cuantización) abordan estos desafíos, y esta herramienta te permite explorar todas ellas.

Con esta comprensión intuitiva, estás listo para explorar las detalles técnicos que siguen. ¡Bienvenido a la frontera del procesamiento de lenguaje!

3. Trabajo Relacionado

Este trabajo se sitúa en la intersección de tres áreas de investigación activas: arquitecturas de atención alternativas, mecanismos de atención dispersa y algoritmos de optimización avanzados. Esta sección sintetiza literatura relevante a través de estos dominios para contextualizar las contribuciones.

3.1. Arquitecturas de Mezclador de Secuencias

La trayectoria del modelado de secuencias en inteligencia artificial ha sido moldeada por una tensión continua entre expresividad computacional y eficiencia de recursos. El advenimiento del mecanismo de atención estándar de transformador transformó fundamentalmente el procesamiento del lenguaje natural, visión por computadora y biología computacional priorizando la contextualización global sobre la recurrencia secuencial. Sin embargo, a medida que la ambición de los modelos fundacionales escala hacia ventanas de contexto de millones de tokens, la complejidad computacional cuadrática $\mathcal{O}(n^2)$ de la autoatención tradicional ha surgido como un cuello de botella severo [35].

3.1.1. Líneas Base de Atención Densa

Atención Softmax Estándar. El mecanismo de autoatención multi-cabeza estándar estableció un cambio de paradigma eliminando completamente la recurrencia secuencial y convoluciones localizadas. Al permitir que cada token en una secuencia atienda directamente a cualquier otro token, el transformador estándar logró un éxito sin precedentes en capturar dependencias de largo alcance. La formulación calcula una suma ponderada de vectores de valor:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

Para la decodificación autorregresiva, este mecanismo requiere almacenamiento en caché de Key-Value (KV) para evitar la recomputación $\mathcal{O}(n^2)$. Sin embargo, esta eficiencia teórica viene con el costo de una huella de memoria que crece linealmente requiriendo espacio $\mathcal{O}(n \cdot d)$, que puede consumir cientos de gigabytes de Memoria de Ancho de Banda Alto (HBM) para modelos grandes [35].

Atención Sigmoide. La atención sigmoide reemplaza la normalización softmax por filas con activación sigmoide elemento por elemento:

$$\text{SigmoidAttn}(Q, K, V) = \sigma\left(\frac{QK^\top}{\sqrt{d_k}} + b\right) V$$

Los análisis teóricos que aprovechan la perspectiva de mezcla de expertos revelan ventajas críticas en complejidad muestral. Para modelos que utilizan expertos ReLU, la compuerta softmax converge a $\mathcal{O}(n^{-0,24})$, mientras que la versión sigmoide logra una convergencia significativamente más rápida $\mathcal{O}(n^{-0,51})$. La naturaleza elemento por elemento evita la sobrecarga de sincronización entre hilos, permitiendo una aceleración del kernel de inferencia del 17 % mediante FlashSigmoid en GPUs NVIDIA H100 [27]. Sin embargo, el escalado empírico reveló inestabilidades de entrenamiento que requieren una capa de estabilización de "norma híbrida" para modelos a gran escala.

3.1.2. Arquitecturas Recurrentes y Retentivas

RetNet (Red Retentiva). RetNet fue diseñada explícitamente para resolver el "triángulo imposible" del modelado de secuencias: lograr entrenamiento paralelizable, inferencia de bajo costo $\mathcal{O}(1)$ y alto rendimiento simultáneamente. El mecanismo de retención introduce una matriz de decadencia exponencial causal fija D:

$$\text{Retention}(X) = (QK^\top \odot D)V$$

con forma recurrente:

$$S_n = \gamma S_{n-1} + k_n^\top v_n, \quad o_n = q_n S_n$$

Esta representación dual permite entrenamiento paralelo con complejidad $\mathcal{O}(n^2 \cdot d)$ e inferencia recurrente con tiempo $\mathcal{O}(1)$ por paso, eliminando completamente el caché KV [33].

Mamba (Modelo de Espacio de Estado Selectivo). Mamba aborda las limitaciones de los modelos de espacio de estado de Tiempo Invariante Lineal (LTI) introduciendo selectividad a los parámetros fundamentales. El sistema de tiempo continuo:

$$h'(t) = Ah(t) + Bx(t), \quad y(t) = Ch(t)$$

se discretiza con parámetros dependientes de entrada:

$$\bar{A}_t = \exp(\Delta_t A), \quad \bar{B}_t = \Delta_t B_t, \quad \Delta_t = \text{softplus}(\text{Linear}(x_t))$$

El algoritmo de escaneo consciente del hardware permite complejidad lineal $\mathcal{O}(n \cdot d)$ con utilización eficiente de GPU, haciendo que la recurrencia lineal sea práctica en aceleradores [10].

Transformadores Continuos Estilo ODE. El transformador ODE trata la profundidad como integración numérica sobre un sistema dinámico continuo:

$$\frac{dh(t)}{dt} = f_\theta(h(t), t)$$

con refinamiento Runge-Kutta que reduce el error de truncamiento y mejora la calidad de generación de secuencias. Esto proporciona una trayectoria intra-bloque más expresiva con uso compartido de pesos, aunque a mayor costo computacional [47].

Atención Aumentada con Memoria (Titans). Titans introduce memorización neuronal en tiempo de prueba: en lugar de almacenar solo activaciones, el modelo actualiza su memoria interna utilizando señales de aprendizaje impulsadas por sorpresa durante la inferencia. Esto aborda el contexto más allá de lo que los modelos recurrentes de estado fijo típicamente soportan, permitiendo contextos extremadamente largos y recuerdo asociativo. El costo del sistema es una mayor complejidad de implementación porque la inferencia ahora incluye lógica de adaptación de memoria [2].

3.1.3. Mecanismos de Atención Dispersa

La complejidad cuadrática de la atención estándar se vuelve prohibitiva para secuencias que exceden cientos de miles de tokens. Los mecanismos de atención dispersa abordan esto restringiendo el conjunto de interacciones de tokens, explotando la observación de que la mayoría de los pesos de atención aprendidos están cerca de cero.

Sparse Transformer. Introduce atención dispersa factorizada reduciendo la complejidad $\mathcal{O}(n^2)$ a $\mathcal{O}(n\sqrt{n})$. Utiliza dos patrones dispersos complementarios: atención con zancada y atención fija, cada una atendiendo a $\mathcal{O}(\sqrt{n})$ posiciones [7].

Longformer. Introduce atención de ventana deslizante con tokens globales opcionales, logrando escalado lineal $\mathcal{O}(n \cdot w)$ donde w es el tamaño de ventana fijo. Combina ventanas deslizantes, patrones dilatados y tokens globales para mantener conectividad de largo alcance con cómputo disperso [3].

BigBird. Combina ventanas locales, enlaces aleatorios y tokens globales para preservar una fuerte conectividad de contexto largo con cómputo disperso. Máscara dispersa aleatoria más rutas locales/globales mantiene propiedades teóricas de aproximación [46].

SparseK. Utiliza una proyección estilo top-k diferenciable sobre puntuaciones de importancia antes de la atención, por lo que solo los pares KV seleccionados participan en la ruta de producto punto costosa [21].

NSA (Atención Dispersa Nativa). Implementa un diseño disperso de tres ramas: rama comprimida, rama seleccionada y rama de ventana local, luego las combina con compuertas aprendidas [44].

SpargeAttn. Filtrado de bloques sin entrenamiento en dos etapas: primero predice interacciones de bloques despreciables, luego aplica poda consciente de softmax para eliminar bloques de baja contribución [48].

FASA. Selección consciente de frecuencia basada en componentes de frecuencia dominantes RoPE, luego aplica atención completa solo en tokens seleccionados [38].

3.1.4. Mecanismos de Memoria con Compuerta

Los mecanismos de memoria con compuerta introducen control dependiente de datos sobre cómo se retiene, olvida y actualiza la información en arquitecturas lineales o recurrentes.

GLA (Atención Lineal con Compuerta). GLA aumenta la atención lineal con una compuerta de decaimiento multiplicativa dependiente de datos. Esto aborda directamente la sobrecarga de memoria en estados recurrentes aditivos, permitiendo que el modelo controle activamente qué información retener a través del tiempo [41].

DeltaNet. DeltaNet reemplaza la acumulación pura con una regla delta: la memoria se corrige comparando el valor recuperado contra el valor deseado. Esto puede interpretarse como corrección de errores en línea y mejora el recuerdo asociativo [42].

Gated DeltaNet. Gated DeltaNet combina ambas ideas: una compuerta de decaimiento controla el olvido mientras que la regla delta controla la escritura dirigida. Esto se presenta como el diseño recurrente lineal más equilibrado porque soporta tanto el borrado rápido como las actualizaciones precisas simultáneamente [43].

HGRN2. HGRN2 utiliza compuertas de olvido jerárquicas acotadas inferiormente con expansión de estado de producto externo. Los límites jerárquicos fomentan que diferentes capas se especialicen en diferentes escalas de tiempo [25].

FoX (Transformador de Olvido). FoX mantiene atención softmax completa pero añade una compuerta de olvido en el espacio de logit. Esto inyecta dinámica consciente de recencia en la canalización estándar sin perder la expresividad de atención completa, aunque mantiene la complejidad cuadrática [17].

Atención con Compuerta después de SDPA. La arquitectura final aplica una compuerta sigmoide después de la atención producto-escala escalada. Los beneficios incluyen la no linealidad adicional que rompe parte del cuello de botella de rango bajo en la ruta de salida, y la compuerta ayuda a suprimir el fenómeno de sumidero de atención [26].

3.2. Algoritmos de Optimización Avanzados

El paisaje de optimización moderna en aprendizaje profundo se ha expandido más allá de SGD y AdamW, con varios desarrollos que abordan desafíos específicos.

Reducción de Varianza. Adan reformula la aceleración Nesterov para aprendizaje profundo, ADOPT corrige problemas teóricos de no convergencia de Adam, y AdEMAMix introduce mezcla de historias duales EMA [39, 34, 24].

Variantes Eficientes en Memoria. Adafactor y GaLore reducen el estado del optimizador mediante factorización o proyección de rango bajo, mientras que Lion utiliza actualizaciones basadas en signos [30, 49, 6].

Enfoques Sin Programador. Schedule-Free AdamW elimina la necesidad de diseño de programador explícito integrando la dinámica de programación en las actualizaciones [8].

Métodos Conscientes de la Curvatura. Sophia, Shampoo y SOAP incorporan información de segundo orden aproximada mediante estimaciones de Hessiano diagonal, estadísticas estructuradas de Kronecker o seguimiento de base propia [12, 36, 18].

Optimizadores Orientados a la Geometría. Muon y Turbo-Muon remodelan explícitamente la geometría de actualización mediante ortogonalización, útil cuando la estructura matricial importa para dar forma a representaciones [31, 4].

4. Arquitectura Centrada en Configuración

4.1. Contrato de Esquema y Validación

El sistema se basa en un contrato de esquema YAML estricto que impone experimentos válidos fallando rápido en configuraciones inválidas. El esquema está estructurado jerárquicamente con `additionalProperties: false` en todos los niveles para eliminar el comportamiento silencioso de "tragar" parámetros desconocidos.

El contrato autoritativo es `schema.yaml`, accesible tanto desde el directorio raíz como desde el directorio `configs/` para facilitar el descubrimiento y la accesibilidad. El esquema impone una estructura de objeto de nivel superior con validación estricta de tipos, rangos y enumeraciones.

Los componentes principales del esquema son:

- **model_class**: Selección de arquitectura (`frankenstein` para codificador de arquitectura mixta, `mini` para variante simplificada, `frankesteindecoder` para decodificador autorregresivo).
- **model**: Arquitectura del modelo (tamaño oculto, capas, tipo de normalización, tipo de mezclador, parámetros específicos del mezclador, modo de atención para encoder/decoder).
- **tokenizer**: Tokenización y vocabulario (tipo, ruta del modelo, ruta del vocabulario, longitud máxima).
- **training**: Configuración de tiempo de ejecución (tamaño de lote, acumulación de gradientes, programador, cortado de gradientes, repetición de datasets, puntos de control).

- **optimizer**: Configuración del optimizador (clase del optimizador, hiperparámetros con prefijo para refinamiento de grupo de parámetros).
- **data**: Carga de datos (directorio de caché, tamaño máximo de secuencia, probabilidad MLM, tipo de datos).
- **deploy**: Opciones de despliegue (configuración de cuantización, ruta del modelo de salida).
- **sbert**: Configuración específica de SBERT (modo, ruta del corpus, hiperparámetros de entrenamiento e inferencia).

4.2. Validación de Esquema

La validación ocurre en la carga de configuración utilizando el validador PyYAML. Las reglas principales incluyen:

- **Validación de tipo**: Todos los campos deben coincidir con los tipos declarados (entero, flotante, booleano, cadena, enumeración).
- **Validación de rango**: Los campos numéricos deben respetar las restricciones mínimas/máximas (por ejemplo, `batch_size` ≥ 1, `learning_rate` > 0).
- **Validación de enumeración**: Los campos de enumeración deben elegir de conjuntos de opciones predefinidos (por ejemplo, `norm_type` $\in$ {`layer_norm`, `dynamic_tanh`, `derf`}).
- **Validación de estructura**: Los objetos anidados deben cumplir con sus esquemas anidados.
- **Validación de propiedades adicionales**: Se rechazan campos no declarados explícitamente.

4.3. Selección de Clase de Modelo

El campo `model_class` determina la variante arquitectónica instanciada por la canalización de entrenamiento. Se soportan tres opciones:

- **frankenstein**: Modelos de codificador de arquitectura mixta que admiten diversos mecanismos de atención (softmax estándar, sigmoide, retentivo, espacio de estado, disperso y mecanismos con compuerta) con MoE (Mezcla de Expertos) y características avanzadas. Optimizado para entrenamiento estilo codificador con objetivo de modelado de lenguaje enmascarado (MLM).
- **mini**: Variante de codificador simplificada diseñada para escenarios de entrenamiento a menor escala. Proporciona sobrecarga reducida de parámetros e iteración más rápida para experimentación y prototipado.
- **frankesteindecoder**: Decodificador autorregresivo causal para generación de siguiente token estilo LLM. Permite enmascaramiento de atención causal para tareas de generación de texto secuencial. Cuando se selecciona esta clase, el tiempo de ejecución impone `mode='decoder'`.

4.4. Selección de Modo de Entrenamiento

El campo `model.mode` controla el comportamiento de enmascaramiento de atención en todo el modelo:

- **encoder**: Utiliza atención bidireccional donde todos los tokens asisten a todos los otros tokens en la secuencia. Adecuado para tareas de modelado de lenguaje enmascarado (MLM) donde el modelo aprende a predecir tokens enmascarados aleatoriamente basado en contexto completo.

- **decoder**: Utiliza enmascaramiento causal donde cada token solo puede asistir a tokens anteriores en la secuencia. Requerido para tareas de predicción autorregresiva de siguiente token como modelado de lenguaje y generación de texto. Cuando `model_class='frankesteindecoder'`, el sistema automáticamente fuerza `mode='decoder'` en tiempo de ejecución.

Este soporte dual de arquitectura permite que el sistema maneje tanto el pre-entrenamiento estilo codificador (MLM en contextos bidireccionales) como la generación estilo decodificador (predicción autorregresiva causal) a través de una interfaz de configuración unificada.

4.5. Tipos de Tarea de Entrenamiento

El campo `training.task` determina el objetivo de entrenamiento, trabajando en conjunto con `model.mode` para definir cómo aprende el modelo:

Modelado de Lenguaje Enmascarado (MLM).

- **Modo**: Requiere `mode='encoder'` para atención bidireccional.
- **Objetivo**: Enmascarar aleatoriamente tokens en la secuencia de entrada (típicamente 15 %) y entrenar el modelo para predecir tokens enmascarados basado en contexto bidireccional completo.
- **Caso de Uso**: Pre-entrenamiento de codificadores para aprendizaje de representación, siguiendo metodología estilo BERT [9]. El modelo aprende representaciones bidireccionales capturando contexto de izquierda y derecha.
- **Configuración**: Utiliza parámetro `mlm_probability` para controlar fracción de enmascaramiento.

Predicción Autorregresiva de Siguiente Token.

- **Modo**: Requiere `mode='decoder'` para enmascaramiento causal.
- **Objetivo**: Entrenar modelo para predecir siguiente token en secuencia dado todos tokens anteriores solamente.
- **Caso de Uso**: Tareas de generación de lenguaje y estilo LLM siguiendo metodología GPT. El modelo aprende dependencias secuenciales con atención causal donde cada token solo puede asistir a tokens precedentes.
- **Clase de Modelo**: Típicamente utiliza `model_class='frankesteindecoder'` para arquitecturas decodificador autorregresivas.

Este soporte dual de tarea permite experimentación unificada a través de tanto pre-entrenamiento estilo codificador (MLM para entendimiento bidireccional) como generación estilo decodificador (AR para producción secuencial de texto) dentro de la misma base de código.

5. Taxonomía de Arquitectura e Implementación

5.1. Familias de Atención y Mezclador de Secuencias

Este sistema implementa diecisiete arquitecturas distintas de mezclador de secuencias organizadas en cinco categorías funcionales que reflejan tendencias de investigación en el diseño de modelado de secuencias. La organización taxonómica refleja la comprensión evolutiva de cómo equilibrar expresividad, eficiencia computacional y restricciones de memoria.

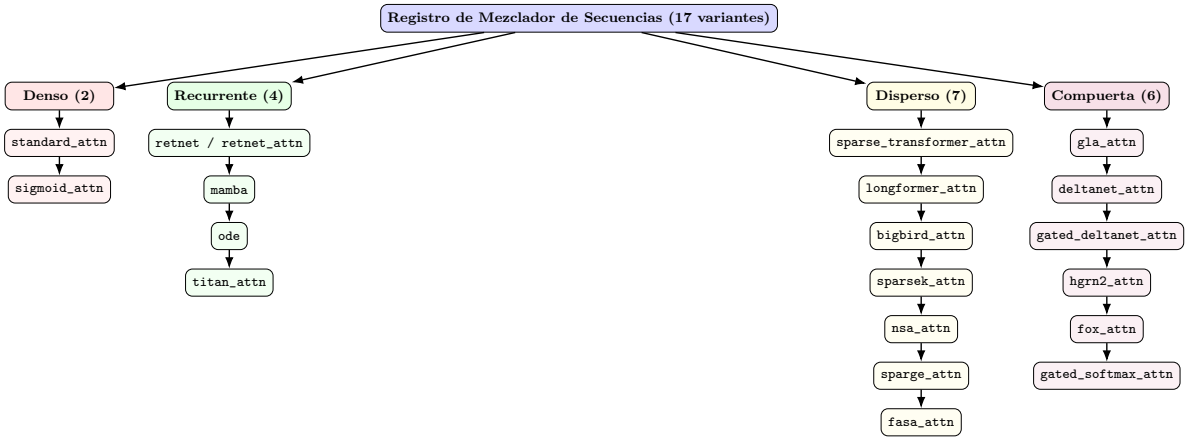

Figura 1: Taxonomía integral de diecisiete arquitecturas de mezclador de secuencias soportadas. Las líneas base densas proporcionan enrutamiento global completo a costo cuadrático. Las arquitecturas recurrentes permiten inferencia de tiempo constante mediante compresión de estado. Las variantes dispersas reducen complejidad mediante patrones estructurados o selección de tokens. Los mecanismos con compuerta introducen control dependiente de datos sobre la retención y olvido de memoria.

1. **Líneas Base de Atención Densa**: Atención softmax estándar y atención sigmoide proporcionan contextualización global completa a costo computacional cuadrático, sirviendo como líneas base de referencia para comparación con alternativas más eficientes.

2. **Arquitecturas Recurrentes y Retentivas**: RetNet, Mamba, bloques estilo ODE y Titans mantienen representaciones de estado que permiten costo de inferencia $\mathcal{O}(1)$ mientras preservan expresividad a través de dinámica recurrente, parámetros selectivos o adaptación de memoria en tiempo de prueba.

3. **Patrones de Atención Dispersa**: Siete variantes dispersas (Sparse Transformer, Longformer, BigBird, SparseK, NSA, SpargeAttn, FASA) reducen la complejidad cuadrática mediante dispersión estructurada, selección de tokens o estrategias de poda sin entrenamiento.

4. **Mecanismos de Memoria con Compuerta**: Seis arquitecturas con compuerta (GLA, DeltaNet, Gated DeltaNet, HGRN2, FoX, Gated Softmax) introducen control dependiente de datos sobre la retención de memoria, el olvido y la fuerza de actualización.

5.2. Atención Estándar

Dadas matrices proyectadas (Q, K, V):

$$\mathrm{Attn}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

Este es el mecanismo de línea base para el enrutamiento de contenido [35].

5.3. Atención Sigmoide

La atención sigmoide elimina la normalización de probabilidad por filas:

$$\mathrm{SigmoidAttn}(Q, K, V) = \sigma\left(\frac{QK^\top}{\sqrt{d_k}} + b\right) V$$

y tiene diferentes requisitos de estabilidad de entrenamiento, a menudo con normalización adicional [27].

5.4. Formulación Retentiva

RetNet utiliza retención con matriz de decaimiento D:

$$\text{Retention}(Q, K, V) = \left(QK^\top \odot D\right) V$$

con forma recurrente:

$$S_n = \gamma S_{n-1} + k_n^\top v_n, \qquad o_n = q_n S_n$$

permitiendo inferencia recurrente de bajo costo [33, 40].

5.5. SSM Selectivo (Mamba)

La recurrencia de espacio de estado selectivo discreto es:

$$h_t = \bar{A}_t h_{t-1} + \bar{B}_t x_t, \qquad y_t = C_t h_t$$

donde $(\bar{A}_t, \bar{B}_t, C_t)$ dependen de la entrada, preservando el escalado de tiempo lineal con escaneo consciente del hardware [10, 13].

5.6. Actualizaciones Continuas Estilo ODE

Enmarcación de profundidad continua:

$$\frac{dh(t)}{dt} = f_\theta(h(t), t)$$

con integradores RK prácticos para ejecución discreta [47].

5.7. Memoria en Tiempo de Prueba (Titans)

Una actualización aumentada con memoria puede escribirse:

$$M_t = (1 - \alpha_t) M_{t-1} + S_t, \qquad S_t = \eta_t S_{t-1} - \theta_t \nabla \ell(M_{t-1}; x_t)$$

para adaptar la memoria en tiempo de inferencia [2, 1].

5.8. Extensiones Dispersas y con Compuerta en la Base de Código Actual

El registro de mezclador ahora incluye bloques dispersos: `sparse_transformer_attn`, `longformer_attn`, `bigbird_attn`, `sparsek_attn`, `nsa_attn`, `sparge_attn`, y `fasa_attn`; y bloques con compuerta: `gla_attn`, `deltanet_attn`, `gated_deltanet_attn`, `hgrn2_attn`, `fox_attn`, y `gated_softmax_attn`.

La implementación impone una política de ejecución explícita para métodos dispersos sin entrenamiento: `fasa_attn` y `sparge_attn` son solo de evaluación/inferencia y lanzan errores de tiempo de ejecución si se usan mientras el modelo está en modo de entrenamiento.

6. Familias de Optimizadores y Dinámica de Entrenamiento

La optimización de arquitecturas de transformador altamente parametrizadas presenta desafíos significativos debido a paisajes de pérdida no convexos, puntos de silla y heterogeneidad de bloques a través de grupos de parámetros. El espectro de Hessiano varía dramáticamente entre incrustaciones, pesos de atención, capas de normalización y redes de retroalimentación, creando caminos de optimización donde ciertas dimensiones son exponencialmente más agudas o planas que otras. Este sistema aborda estos desafíos a través de un marco unificado que admite veinte familias de optimizadores que abarcan múltiples trayectorias algorítmicas.

6.1. Organización Taxonómica de Optimizadores Soportados

El soporte de optimizadores está organizado en seis categorías algorítmicas que reflejan direcciones de investigación modernas en optimización de aprendizaje profundo:

1. **Líneas Base Clásicas**: SGD con momentum, Adam y AdamW definen el suelo de comparación contra el cual se evalúan métodos más nuevos. Estos métodos son bien entendidos pero exhiben limitaciones incluyendo sensibilidad a programadores de tasa de aprendizaje, robustez pobre a curvatura heterogénea (SGD) y alta sobrecarga de memoria para modelos grandes (familia Adam).

2. **Momentum Avanzado y Reducción de Varianza**: Adan, ADOPT, AdEMAMix, MARS y Cautious AdamW (2024–2025) abordan limitaciones específicas de métodos clásicos. Adan reformula la aceleración Nesterov, ADOPT corrige la no convergencia teórica de Adam, AdEMAMix introduce mezcla de historias duales EMA, MARS trae reducción de varianza al entrenamiento a gran escala, y variantes Cautious enmascaran actualizaciones a direcciones consistentes en signo.

3. **Optimizadores Eficientes en Memoria**: Adafactor, GaLore y Lion reducen el estado del optimizador mediante factorización, proyección de rango bajo o actualizaciones basadas en signos. Estos métodos son críticos cuando VRAM está dominado por estado del optimizador más que activaciones.

4. **Sin Programador y Sin Parámetros**: Schedule-Free AdamW y Prodigy eliminan el diseño de programador explícito o ajuste de tasa de aprendizaje haciendo que la dinámica de actualización absorba estas responsabilidades, reduciendo la complejidad de búsqueda de hiperparámetros.

5. **Conscientes de la Curvatura y de Segundo Orden**: Sophia, Shampoo y SOAP incorporan información de segundo orden aproximada mediante estimaciones de Hessiano diagonal, estadísticas estructuradas de Kronecker o seguimiento de base propia, mejorando la condición a costo de complejidad de implementación.

6. **Orientados a Geometría**: Muon y Turbo-Muon remodelan explícitamente la geometría de actualización mediante ortogonalización, útil cuando la estructura matricial importa para dar forma a representaciones.

6.2. Formulación Adaptativa Central

Muchos optimizadores soportados comparten formulación fundamental de seguimiento de momentos. Los optimizadores de familia Adam rastrean primer y segundo momentos de gradientes:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$$

seguidos por actualizaciones precondicionadas con corrección de sesgo:

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t}, \quad \theta_{t+1} = \theta_t - \eta_t \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda \theta_t \right)$$

donde η_t es el programador de tasa de aprendizaje, ϵ es constante de estabilidad numérica, y λ es coeficiente de decaimiento de pesos [20].

7. Cuantización y Despliegue

La pila de despliegue utiliza empaquetado de pesos ternarios más cuantización de activaciones INT8 para artefactos eficientes.

7.1. Cuantización Ternaria

Dado tensor de pesos W, un escalado práctico es:

$$s = \text{mean}(|W|), \qquad \tilde{W} = \text{clip}\left(\text{round}\left(\frac{W}{s}\right), -1, 1\right)$$

que aproxima actualizaciones de bajo bits estilo BitNet [37]. La mapeo empaquetada utiliza dos bits por símbolo de peso para eficiencia de almacenamiento.

7.2. Cuantización de Activaciones

Para activaciones x:

$$q = \text{round}\left(x \cdot \frac{127}{\text{máx}(|x|) + \epsilon}\right), \qquad q \in [-128, 127]$$

con decuantización $x \approx q/\alpha$.

7.3. Estimaciones de Tamaño

Para N parámetros:

$$\text{Tamaño FP32} \approx 4N, \quad \text{Tamaño FP16} \approx 2N, \quad \text{Tamaño 1.58-bit} \approx \frac{1{,}58}{8}N$$

antes de metadatos y sobrecarga de empaquetado. Esto se alinea con objetivos de despliegue ligero [29, 5].

8. Tareas Posteriores SBERT

La incrustación de oraciones se basa en entrenamiento estilo siamés [28]. Para par de oraciones (s_1, s_2) con incrustaciones (e_1, e_2):

$$\cos(e_1, e_2) = \frac{e_1^\top e_2}{\|e_1\|\|e_2\|}$$

y pérdida de coseno estilo regresión:

$$\mathcal{L}_{\cos} = (\cos(e_1, e_2) - y)^2$$

con $y \in [-1, 1]$ en esta canalización.

Modos posteriores soportados:

- **Similitud**: puntuación por pares entre dos oraciones.
- **Búsqueda**: k vecinos más cercanos sobre un corpus.
- **Agrupamiento**: agrupación de incrustaciones (por ejemplo, k-means).
- **Codificación**: exportación de incrustaciones persistentes para recuperación posterior.

9. Tablas Resumen

9.1. Resumen de Atención y Mezclador de Secuencias

Tipo	Ecuación Central	Entrenamiento	Inferencia	Notas
Atención Estándar	$\mathrm{softmax}(QK^\top/\sqrt{d_k})V$	$\mathcal{O}(n^2 d)$	$\mathcal{O}(n)$/paso	Enrutamiento global expresivo de línea base [35].
Atención Sigmoide	$\sigma(QK^\top/\sqrt{d_k}+b)V$	$\mathcal{O}(n^2 d)$	$\mathcal{O}(n)$/paso	Compuerta elemento por elemento; a menudo necesita norma de estabilización [27].
RetNet	$(QK^\top \odot D)V$	$\mathcal{O}(n^2 d)$ o por bloques	$\mathcal{O}(1)$/paso	Forma dual paralela/recurrente con retención de decaimiento [33].
Mamba	$h_t = \bar{A}_t h_{t-1} + \bar{B}_t x_t$	$\mathcal{O}(nd)$	$\mathcal{O}(1)$/paso	Espacio de estado selectivo con escaneo consciente del hardware [10].
Bloque estilo ODE	$\frac{dh}{dt} = f_\theta(h,t)$	depende del solucionador	depende del solucionador	Interpretación de profundidad continua; integración RK [47].
Memoria Titans	$M_t = (1-\alpha_t)M_{t-1} + S_t$	aprox. $\mathcal{O}(nd)$	centrada en recuperación	Actualizaciones de memoria en tiempo de prueba con dinámica impulsada por sorpresa [2].

9.2. Resumen de Optimizadores

Optimizador	Familia	Costo de Estado	Idea Clave	Ref
AdamW	Adaptativo momento primero/segundo	Alto	Línea base con decaimiento de pesos desacoplado	[20]
RAdam	Adaptativo corregido en varianza	Alto	Rectifica varianza adaptativa temprana	[19]
Adan	Momentum + reducción de varianza	Alto	Actualización adaptativa estilo Nesterov	[39]
ADOPT	Variante de Adam	Alto	Actualizaciones reordenadas con garantías de convergencia mejoradas	[34]
AdEMAMix	Adaptativo multi-EMA	Alto	Mezcla EMAs de horizonte corto y largo	[24]
MARS	Precondicionado reducido en varianza	Alto	Corrección de momentum recursiva	[45]
Cautious AdamW	Momentum enmascarado	Alto	Aplica actualizaciones solo en direcciones consistentes en signo	[16]
Schedule-free AdamW	Adaptativo sin programador	Alto	Elimina dependencia de programador LR explícito	[8]
Adafactor	Adaptativo eficiente en memoria	Medio	Segundos momentos factorizados para tensores de matriz	[30]
GaLore AdamW	Proyección de gradiente de rango bajo	Medio	Optimiza en espacio de gradiente proyectado de rango bajo	[49]
Prodigy	Adaptación sin parámetros	Medio	Calibración de paso adaptativa a distancia	[22]

Optimizador	Familia	Costo de Estado	Idea Clave	Ref
Lion	Momentum de signo	Bajo	Actualización de momentum de signo, estado reducido	[6]
Sophia	Segundo orden aprox.	Medio	Precondicionamiento de Hessiano diagonal con recorte	[18]
Shampoo	Precondicionador de matriz	Alto	Estadísticas de segundo orden estructuradas de Kronecker	[12]
SOAP	Shampoo + base Adam	Alto	Seguimiento estilo Adam en base de precondicionador	[36]
Muon	Basado en ortogonalidad	Medio	Actualizaciones matriciales ortogonalizadas	[31]
Turbo-Muon	Ortogonalización acelerada	Medio	Aceleración de Newton-Schulz precondicionada	[4]

10. Discusión

Las elecciones de diseño en Transformer Encoder Frankenstein reflejan varias tensiones de ingeniería e investigación en herramientas modernas de aprendizaje profundo.

10.1. Compromisos de Diseño Impulsado por Esquemas

El enfoque basado primero en esquemas proporciona beneficios significativos de reproducibilidad imponiendo contratos explícitos y fallando rápido en configuraciones inválidas. Sin embargo, este enfoque también introduce rigidez: agregar nuevas arquitecturas u optimizadores requiere extensiones de esquema más que argumentos de línea de comandos sueltos. El sistema de hiperparámetros con prefijo permite control granular pero aumenta la complejidad de configuración para usuarios acostumbrados a interfaces más simples.

La decisión de imponer `additionalProperties: false` en todos los niveles de esquema elimina el comportamiento silencioso de "tragar" parámetros que ha plagado sistemas de configuración anteriores, pero esta estrictitud requiere mantenimiento cuidadoso del esquema al extender capacidades del sistema. Cada nuevo mecanismo de atención o variante de optimizador debe integrarse adecuadamente en el marco de validación, incluyendo definiciones de campos de esquema con tipos y restricciones apropiados, mapeo de hiperparámetros con prefijo para grupos específicos de optimizador, valores predeterminados alineados con mejores prácticas de investigación, y cadenas de documentación para renderizado de interfaz web.

10.2. Cobertura Arquitectónica y Brechas

Las diecisiete arquitecturas de mezclador implementadas abarcan direcciones principales de investigación en modelado de secuencias, pero ciertas brechas permanecen. El sistema carece de arquitecturas híbridas recientes como Griffin [32] y Jamba [11], que combinan compuertas con modelos de espacio de estado. El enrutamiento MoE (Mezcla de Expertos) está implementado para capas FFN pero no para cómputo de atención, donde el trabajo reciente ha mostrado beneficios [15].

La cobertura de atención dispersa es integral, pero la implementación de métodos sin entrenamiento (FASA, SpargeAttn) lanza errores de tiempo de ejecución durante el entrenamiento, reflejando restricciones arquitectónicas: estos métodos requieren puntos de control preentrenados de modelos de atención completa o procedimientos de ajuste específicos que no están automatizados actualmente.

La cobertura de mecanismo con compuerta es fuerte a través de categorías principales (GLA, DeltaNet, Gated DeltaNet, HGRN2, FoX, Gated Softmax).

10.3. Fragmentación del Paisaje de Optimizadores

El soporte para veinte familias de optimizadores a través de seis categorías algorítmicas demuestra integralidad pero también destaca el estado fragmentado de investigación de optimización. Los usuarios enfrentan una complejidad de decisión significativa al elegir entre métodos de reducción de varianza (Adan, MARS), variantes eficientes en memoria (GaLore, Adafactor) y enfoques conscientes de la curvatura (Sophia, Shampoo). El sistema de hiperparámetros con prefijo, aunque poderoso, requiere comprensión de qué parámetros son relevantes para cada clase de optimizador.

La calidad de implementación varía a través de optimizadores: métodos clásicos (AdamW, SGD con momentum) están altamente optimizados en PyTorch, mientras que métodos más nuevos (Muon, Turbo-Muon, SOAP) pueden requerir implementaciones personalizadas que afectan estabilidad numérica y características de rendimiento.

10.4. Consideraciones de Despliegue y Producción

La canalización de cuantización demuestra preocupaciones prácticas de despliegue pero hace compromisos de ingeniería específicos. El empaquetado de pesos ternarios reduce el almacenamiento a aproximadamente 1,58 bits por parámetro, pero esta compresión agresiva puede degradar el rendimiento, especialmente para modelos más pequeños donde el error de cuantización es más significativo. La implementación actual aplica cuantización uniformemente a través de tipos de parámetros.

Los flujos de trabajo SBERT proporcionan utilidad práctica para tareas de similitud semántica y recuperación, pero la implementación asume estrategias de agrupamiento estándar (token CLS, agrupamiento promedio). Avances recientes como incrustaciones Matryoshka [23] y refinamientos de aprendizaje contrastivo [14] no están incorporados todavía.

10.5. Desafíos de Integración y Extensibilidad

La estructura de base de código actual, aunque funcional, presenta desafíos de mantenimiento a medida que se expanden familias de arquitectura y optimizador. El patrón de despachador para selección de mezclador y enrutamiento de optimizador maneja extensibilidad pero arriesga convertirse en un "fregadero de cocina" de lógica condicional. Versiones futuras se beneficiarían de arquitecturas basadas en complementos donde nuevos mezcladores y optimizadores podrían registrarse declarativamente más que modificar lógica de despacho central.

La interfaz de configuración web proporciona mejoras significativas de usabilidad pero introduce complejidad de despliegue: ejecutar Streamlit junto con trabajos de entrenamiento requiere recursos adicionales e consideraciones de infraestructura que pueden no ser apropiadas para todos los entornos, particularmente clústeres HPC sin acceso web.

11. Conclusión

Transformer Encoder Frankenstein presenta una plataforma de experimentación unificada basada en configuración que aborda desafíos críticos en investigación moderna de aprendizaje profundo: fragmentación arquitectónica a través de atención densa, modelos recurrentes, patrones dispersos y mecanismos con compuerta; complejidad del paisaje de optimizadores que abarca líneas base clásicas, métodos de reducción de varianza, variantes eficientes en memoria, enfoques sin programador, algoritmos conscientes de la curvatura y métodos orientados a geometría; y flujos de trabajo de extremo a extremo que abarcan cuantización y aplicaciones de incrustación de oraciones.

Las contribuciones principales del sistema son:

1. **Diseño Basado Primero en Esquemas**: Un contrato de configuración basado en YAML estricto con validación y enrutamiento de hiperparámetros con prefijo que permite experimentos reproducibles a través de diecisiete arquitecturas de mezclador y veinte familias de optimizadores.

2. **Soporte de Arquitectura Integral**: Implementación que abarca categorías principales de investigación incluyendo líneas base densas (atención estándar, sigmoide), alternativas recurrentes (RetNet, Mamba, bloques estilo ODE, memoria aumentada con memoria), patrones dispersos (Sparse Transformer, Longformer, BigBird, SparseK, NSA, SpargeAttn, FASA) y mecanismos con compuerta (GLA, DeltaNet, Gated DeltaNet, HGRN2, FoX, Gated Softmax).

3. **Marco de Enrutamiento de Optimizadores**: Sistema de grupos de hiperparámetros con prefijo que permite control por grupo de parámetros a través de veinte optimizadores que abarcan líneas base clásicas, reducción de varianza, eficiencia de memoria, programación libre, segundo orden y métodos orientados a geometría.

4. **Cuantización y Despliegue**: Canalización integrada que soporta empaquetado de pesos ternarios y cuantización de activaciones INT8 con estimaciones de tamaño para despliegue ligero.

5. **Flujos de Trabajo de Incrustación de Oraciones**: Canalizaciones de entrenamiento e inferencia inspiradas en SBERT que soportan similitud, recuperación, agrupamiento y exportación de incrustaciones.

6. **Interfaz de Configuración Interactiva**: Servidor web basado en Streamlit que proporciona generación de formularios impulsada por esquemas, validación en tiempo real, documentación en línea y generación de comandos CLI.

Este trabajo demuestra que un diseño basado en esquemas riguroso puede unificar experimentación a través de arquitecturas y optimizadores diversos mientras mantiene reproducibilidad. El sistema proporciona una plataforma sólida para exploración sistemática de diseño de transformadores, permitiendo a investigadores y profesionales iterar rápidamente en configuraciones experimentales sin comprometer condiciones reproducibles.

El desarrollo futuro se centrará en extender la cobertura arquitectónica (agregando variantes híbridas y mecanismos MoE para atención), mejorar la canalización de despliegue (soportando estrategias de cuantización más sofisticadas y métodos de inferencia eficientes), y refinar la interfaz de configuración (añadiendo asistentes guiados para selección de arquitectura/optimizador y exportación de configuraciones compartibles).

A. Arquitecturas Fundamentales de Transformadores: Análisis Teórico y Empírico Integral

La trayectoria del modelado de secuencias en inteligencia artificial ha sido inexorablemente moldeada por una tensión continua entre expresividad computacional y eficiencia de recursos. El advenimiento del mecanismo de atención estándar transformó fundamentalmente el procesamiento del lenguaje natural, visión por computadora y biología computacional al priorizar la contextualización global sobre la recurrencia secuencial y las convoluciones localizadas. Sin embargo, a medida que la ambición de los modelos fundacionales escala hacia ventanas de contexto de millones de tokens—esencial para análisis genómico, comprensión de código a nivel de repositorio y planificación agéntica a largo plazo—la complejidad computacional cuadrática de la autoatención tradicional ha emergido como un cuello de botella severo. La dependencia de la industria en el almacenamiento en caché de Claves-Valores (KV) durante la decodificación

autorregresiva ha expuesto aún más las limitaciones de las arquitecturas estándar, precipitando una crisis de ancho de banda de memoria en los aceleradores de hardware moderno.

En respuesta a estas limitaciones teóricas y de hardware, la comunidad investigadora ha propuesto una proliferación de arquitecturas alternativas. Estos modelos intentan reconciliar el "triángulo imposible" del modelado de secuencias: lograr entrenamiento paralelizable, inferencia de tiempo constante y rendimiento predictivo incompromisible simultáneamente. Esta sección proporciona un análisis comparativo profundo de seis arquitecturas pivotales de modelado de secuencias: el mecanismo de Atención Estándar, Atención Sigmoide, la Red Retentiva (RetNet), el Modelo Selectivo de Espacio de Estado (Mamba), el Transformador de Ecuación Diferencial Ordinaria (ODE), y la arquitectura de Memoria Neural Titans. Deconstruyendo sus formulaciones matemáticas, analizando sus complejidades computacionales, y evaluando sus implicaciones de segundo y tercer orden en utilización de hardware y aprendizaje de representaciones, esta sección establece un marco comprehensivo para entender el futuro del modelado de secuencias.

A.1. Atención Estándar: La Fundamentación de la Contextualización Global

El mecanismo de autoatención multi-cabeza estándar estableció un cambio de paradigma dispensando completamente con la recurrencia secuencial y convoluciones localizadas que caracterizaban las redes de Memoria Corta a Largo Plazo (LSTM) anteriores. Al permitir que cada token en una secuencia atienda directamente a cualquier otro token, el transformador estándar logró un éxito sin precedentes en capturar dependencias de largo alcance y ejecutar tareas de razonamiento complejo basado en contenido.

A.1.1. Formulación Matemática y Mecanismo de Enrutamiento de Tokens

En el núcleo del bloque transformador estándar se encuentra el mecanismo de atención de producto punto escalado. Los tokens de entrada se mapean a incrustaciones de alta dimensionalidad y subsecuentemente se proyectan en matrices de Consulta ($\mathbf{Q}$), Clave ($\mathbf{K}$), y Valor ($\mathbf{V}$) a través de transformaciones lineales aprendidas. La operación de atención se formula matemáticamente para computar una suma ponderada de los vectores de valor.

$$\text{Atención}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_{in}}}\right)\mathbf{V}$$

Para un token específico i en una secuencia autorregresiva, la salida $\mathbf{y}_i$ se computa como una combinación convexa de todos los vectores de valor precedentes. Los pesos se determinan por las similitudes normalizadas entre la consulta actual y todas las claves históricas:

$$\mathbf{y}_i = \sum_{j=1}^{i} \frac{\exp(\mathbf{Q}_i^\top \mathbf{K}_j / d_{in})\mathbf{V}_j}{\sum_{\ell=1}^{i} \exp(\mathbf{Q}_i^\top \mathbf{K}_\ell / d_{in})}$$

La utilización de la función softmax es una decisión arquitectónica crítica que impone una distribución de probabilidad sobre la secuencia. Esto crea un mecanismo de enrutamiento competitivo donde los tokens deben competir por una cantidad finita de "masa de atención". Aunque este mecanismo excele al permitir que la red se enfoque agudamente en tokens altamente relevantes—formando la base de "cabezas de inducción" que impulsan el aprendizaje en contexto—introduce una dependencia estructural global que prohíbe el procesamiento de tokens independiente a nivel de elemento durante el pase hacia adelante.

A.1.2. Complejidad Computacional y el Cuello de Botella del Caché KV

La dependencia matemática de interacciones pairwise densas produce una complejidad computacional y de memoria que escala cuadráticamente con la longitud de secuencia n. Específicamente, la complejidad de tiempo está acotada por $\mathcal{O}(n^2 \cdot d)$ y la complejidad de espacio por

$\mathcal{O}(n^2)$ durante la fase de entrenamiento. Aunque esta operación es altamente paralelizable a través de la dimensión de secuencia en GPUs modernas, la expansión cuadrática permanece como la barrera primaria de entrenar modelos en secuencias que exceden cientos de miles de tokens.

Durante la inferencia autorregresiva, el mecanismo de atención procesa un token a la vez. Para evadir la recomputación redundante $\mathcal{O}(n^2)$ de estados pasados para cada nuevo token, los transformadores en producción emplean un caché de Claves-Valores (KV). La fase de prefill procesa el prompt inicial y puebla el caché, mientras que la fase de generación aprovecha este caché para lograr complejidad $\mathcal{O}(n)$ por paso de decodificación. Sin embargo, esta eficiencia teórica en tiempo viene al costo de una huella de memoria linealmente creciente, requiriendo espacio $\mathcal{O}(n \cdot d)$.

El caché KV induce ineficiencias sistémicas masivas en despliegues a gran escala. Para un modelo con docenas de cabezas de atención y capas, el caché puede consumir fácilmente cientos de gigabytes de Memoria de Ancho de Banda Alto (HBM), severamente restringiendo el tamaño máximo de lote concurrente. Literatura reciente identifica esto como el problema de la "escritura indiscriminada". Porque el modelo debe comprometer cada token generado con su clave y valor al caché independientemente de su utilidad futura, los sistemas rápidamente agotan recursos de memoria. Para mitigar esto, investigadores han explorado Selección KV (lectura selectiva del caché en tiempo de ejecución), Evicción KV (poda retrospectiva de tokens innecesarios), y Admisión KV (predicción de utilidad futura previa al almacenamiento), aunque estos permanecen como aproximaciones heurísticas del mecanismo exacto de atención.

A.1.3. Perfil Arquitectónico: Atención Estándar

Atributo	**Especificación**
Nomenclatura	Atención Estándar (Transformador)
Autores / Año	Vaswani et al. / 2017
Papel / DOI	Attention Is All You Need / 10.48550/arXiv.1706.03762
Complejidad de Entrenamiento	Tiempo: $\mathcal{O}(n^2 \cdot d)$, Espacio: $\mathcal{O}(n^2)$
Complejidad de Inferencia	Tiempo por paso: $\mathcal{O}(n)$, Espacio: $\mathcal{O}(n)$ (con Caché KV explícito)
Ventajas	Expresividad incomparable; recuerdo histórico perfecto a través de la ventana de contexto; fase de entrenamiento altamente paralelizable.
Desventajas	Los cuellos de botella cuadráticos de entrenamiento severamente limitan escalado de longitud de contexto; la huella masiva del caché KV durante decodificación autorregresiva degrada el throughput.
Características	Enrutamiento de tokens competitivo basado en softmax; interacciones contextuales globales densas; codificaciones de posición absolutas o relativas.

A.2. Atención Sigmoide: Localidad Algorítmica Consciente del Hardware

A.2.1. Fundamentación Matemática y Perspectiva de Mezcla de Expertos

La Atención Sigmoide computa los logits de afinidad pre-atención $\mathbf{L} = \mathbf{Q}\mathbf{K}^\top$ idénticamente a la atención estándar. Sin embargo, aplica una función sigmoide no normalizada independientemente a cada logit en lugar de normalizar a través del eje de secuencia. La formulación se define como:

$$\text{AtenciónSigmoide}(\mathbf{L}) = \sigma(\mathbf{L} + \mathbf{b})\mathbf{V}$$

donde $\mathbf{b}$ representa un término de sesgo aprendido o fijo. A diferencia de softmax, que requiere una operación de reducción por fila—específicamente una resta de máximo para estabilidad

numérica seguida de una sumatoria a través de toda la longitud de secuencia para asegurar que las salidas sumen a uno—la función sigmoide evalúa independientemente en cada elemento de la matriz de logit.

Análisis teóricos que aprovechan una perspectiva de Mezcla de Expertos (MoE) revelan ventajas críticas en complejidad muestral para autoatención sigmoide. Modelando las cabezas de atención como expertos gobernados por un mecanismo de compuerta, investigadores evaluaron tasas de convergencia empírica de la pérdida de Voronoi. Para modelos MoE utilizando expertos ReLU, el mecanismo de compuerta softmax cuadrático convergió a una tasa de $\mathcal{O}(n^{-0,24})$. Inversamente, la versión sigmoide logró una tasa de convergencia significativamente más rápida de $\mathcal{O}(n^{-0,51})$. Con expertos lineales, sigmoide obtuvo $\mathcal{O}(n^{-0,46})$ comparado contra $\mathcal{O}(n^{-0,07})$ de softmax.

La naturaleza elemento a elemento de la activación sigmoide mitiga la "competencia de tokens" inherente a softmax. Porque softmax estrictamente acota la masa de probabilidad total a 1.0, una cabeza de atención no puede simultáneamente asignar una puntuación de importancia absoluta alta a múltiples tokens distintos sin diluir las puntuaciones de otros. La atención sigmoide permite a un token afirmar una magnitud alta de relevancia a múltiples tokens de contexto independientemente, habilitando una medida más absoluta de importancia semántica.

A.2.2. Estabilización y el Requisito de Norma Híbrida

A pesar de su superioridad teórica en convergencia, el escalado empírico de atención sigmoide reveló inestabilidades sustanciales de entrenamiento. Durante las etapas tempranas de entrenamiento de modelos de lenguaje grandes (e.g., 1B parámetros con longitud de contexto de 4096), la atención sigmoide sufre de normas de atención masivas iniciales, induciendo picos de gradiente severos que pueden descarrilar la trayectoria de optimización.

Para establecer exitosamente la atención sigmoide como reemplazo drop-in para softmax, investigadores introdujeron la modificación arquitectónica de "norma híbrida". Norma híbrida es una Normalización de Capa adicional aplicada directamente a la salida de la operación de atención antes de la conexión residual:

$$\mathbf{x}_{out} = \mathbf{x} + \text{norm}\left(\sigma\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_{qk}}}\right)\mathbf{V}\right)$$

Esta capa de normalización adicional efectivamente amortigua la magnitud sin restricciones de las activaciones sigmoide, estabilizando el flujo de gradientes a escala y permitiendo al modelo igualar o ligeramente superar líneas base de softmax en evaluaciones downstream.

A.2.3. Complejidad Computacional y Optimización de Hardware

Desde una perspectiva puramente matemática, el número de operaciones de punto flotante (FLOPs) requeridas para atención sigmoide es esencialmente idéntico a la atención softmax. Softmax requiere resta de máximo, exponenciación, sumatoria y división; sigmoide requiere suma de sesgo, inversión de signo, exponenciación, adición y división. Ambas mantienen complejidad teórica de tiempo $\mathcal{O}(n^2 \cdot d)$.

Sin embargo, el diseño de algoritmos no puede divorciarse de la topología de hardware. Porque sigmoide es un mapeo elemento a elemento, completamente circunvala la sobrecarga de sincronización entre hilos requerida por las operaciones de reducción de softmax. Esta localidad algorítmica habilita implementaciones altamente optimizadas conscientes del hardware. La introducción de FlashSigmoide capitaliza en esta independencia, permitiendo a la GPU procesar bloques de memoria completamente dentro de la SRAM ultra-rápida sin vaciar estados intermedios a HBM. Esto genera una aceleración notable del kernel de inferencia del 17 % sobre el marco altamente optimizado de FlashAttention-2 en GPUs NVIDIA H100.

A.2.4. Perfil Arquitectónico: Atención Sigmoide

Atributo	Especificación
Nomenclatura	Autoatención Sigmoide
Autores / Año	Ramapuram et al. (Apple) / 2024–2025
Papel / DOI	Theory, Analysis, and Best Practices for Sigmoid Self-Attention / 10.48550/arXiv.2409.04431
Complejidad de Entrenamiento	Tiempo: $\mathcal{O}(n^2 \cdot d)$, Espacio: $\mathcal{O}(n^2)$ (idéntico a Softmax teóricamente)
Complejidad de Inferencia	Tiempo por paso: $\mathcal{O}(n)$, Espacio: $\mathcal{O}(n)$
Ventajas	Elimina la competencia de tokens con suma cero; regularidad Lipschitz superior; aceleración masiva del kernel de hardware del 17 % vía FlashSigmoide.
Desventajas	Susceptible a picos de gradiente en entrenamiento temprano; requiere modificaciones arquitectónicas como "norma híbrida" para estabilidad a longitudes de contexto grandes.
Características	Activación continua elemento a elemento; ventajas de complejidad muestral respaldadas por MoE; circunvalación completa de sincronización entre filas.

A.3. RetNet: La Dualidad de Recurrencia y Atención

La Red Retentiva (RetNet) fue explícitamente engineered para resolver lo que investigadores términaron el "triángulo imposible" del modelado de secuencias. Históricamente, arquitecturas podían seleccionar dos de tres rasgos ideales: entrenamiento paralelo, inferencia de bajo costo $\mathcal{O}(1)$, y alto rendimiento. Los transformadores logran paralelismo y rendimiento pero fallan en inferencia de bajo costo; RNNs lineales logran inferencia eficiente pero luchan con entrenamiento paralelo y rendimiento. RetNet introduce una arquitectura capaz de soportar los tres simultáneamente.

A.3.1. Fundamentación Matemática y el Mecanismo de Retención Multi-Escala

RetNet completamente reemplaza el mecanismo de autoatención softmax multi-cabeza con un nuevo módulo de retención multi-escala. La innovación arquitectónica está predicada en establecer una dualidad matemática rigurosa entre recurrencia de secuencias y autoatención.

Para incrustar dinámicamente información de posición relativa en las representaciones, RetNet mapea los vectores de consulta y clave al plano complejo utilizando la fórmula de Euler. La transformación incorpora una rotación de vector que hace las incrustaciones intrínsecamente conscientes de posición, denotada como Θ.

El mecanismo de retención introduce una matriz de decadencia exponencial causal fija $\mathbf{D} \in \mathbb{R}^{n \times n}$. En su representación paralela, la salida se computa vía un producto de Hadamard elemento a elemento ($\odot$) aplicado después de la multiplicación de matrices:

$$\text{Retención}(\mathbf{X}) = (\mathbf{Q}\mathbf{K}^\top \odot \mathbf{D})\mathbf{V}$$

donde la matriz de decadencia se define tal que $\mathbf{D}_{nm} = \gamma^{n-m}$ para $n \geq m$, y 0 en otro caso. El escalar de decadencia $\gamma \in (0, 1)$ actúa como un factor de descuento temporal, progresivamente atenuando la influencia de tokens históricos distantes a favor de tokens locales y recientes. RetNet utiliza un enfoque multi-escala, asignando diferentes tasas de decadencia (e.g., $\gamma = 1 - 2^{-5}, \dots$) a diferentes cabezas de retención para capturar tanto dependencias a corto plazo como arcos semánticos de largo alcance.

Crucialmente, porque la matriz de decadencia $\mathbf{D}$ se construye desde un término exponencial simple, la ecuación paralela puede ser algebraicamente transformada en una representación recurrente exacta:

$$\mathbf{S}_n = \gamma \mathbf{S}_{n-1} + \mathbf{K}_n^\top \mathbf{V}_n$$

$$\text{Retención}(\mathbf{X}_n) = \mathbf{Q}_n \mathbf{S}_n$$

En esta formulación, $\mathbf{S}_n$ opera como un estado oculto de tamaño fijo y valuado en matrices que recursivamente acumula el contexto histórico. El modelo computa la interacción consulta-clave implícitamente dentro del vector de estado, completamente eliminando la necesidad de materializar la matriz de atención $n \times n$ o almacenar un caché KV linealmente creciente.

Para optimizar el procesamiento de secuencias excepcionalmente largas, RetNet introduce una representación recurrente híbrida por fragmentos. La secuencia de entrada se divide en segmentos o fragmentos localizados. Dentro de cada fragmento, el modelo computa la retención en paralelo para aprovechar los núcleos de multiplicación de matrices de la GPU. Simultáneamente, un vector de estado recurrente se pasa entre los fragmentos para propagar información de largo alcance.

A.3.2. Complejidad Computacional y Escalado de Rendimiento

El diseño multi-paradigma de RetNet resulta en complejidad computacional dinámica basada en el modo operacional. Durante la fase de entrenamiento, la representación paralela requiere tiempo $\mathcal{O}(n^2 \cdot d)$, idéntico a transformadores estándar. Sin embargo, al desplegar el modo recurrente por fragmentos durante el entrenamiento, esta complejidad se reduce a $\mathcal{O}(n \cdot c \cdot d)$, donde c es el tamaño de fragmento definido, logrando escalado lineal con respecto a la longitud total de secuencia n.

La ventaja más dramática ocurre durante la inferencia autorregresiva. Mediante cambio seamless a la representación recurrente, RetNet opera con complejidad de tiempo $\mathcal{O}(1)$ por paso de decodificación. La complejidad de memoria está estrictamente acotada por $\mathcal{O}(d^2)$ para almacenar la matriz de estado constante $\mathbf{S}_n$. Benchmarks empíricos revelan que una RetNet de parámetros de 6.7B drásticamente supera un Transformador estándar en throughput de decodificación (aumento de 8.4x) y latencia (reducción de 15.6x) mientras utiliza 3.4x menos memoria de GPU a una longitud de contexto de 8k.

A.3.3. Perfil Arquitectónico: RetNet

Atributo	Especificación
Nomenclatura	RetNet (Red Retentiva)
Autores / Año	Sun et al. (Microsoft Research) / 2023
Papel / DOI	Retentive Network: A Successor to Transformer for Large Language Models / 10.48550/arXiv.2307.08621
Complejidad de Entrenamiento	Tiempo: $\mathcal{O}(n \cdot c)$ (por fragmentos) o $\mathcal{O}(n^2)$ (paralela)
Complejidad de Inferencia	Tiempo por paso: $\mathcal{O}(1)$, Espacio: $\mathcal{O}(1)$ (matriz de estado constante)
Ventajas	Elimina completamente el caché KV; inferencia altamente eficiente de $\mathcal{O}(1)$; capacidad de entrenamiento paralelo; resuelve el triángulo imposible.
Desventajas	La decadencia exponencial impone un sesgo estructural inductive rígido que puede artificialmente truncar dependencias semánticas de largo alcance comparado con atención softmax.
Características	Tri-paradigma de computación (Paralela, Recurrente, Por Fragmentos); codificación de posición en plano complejo; factores de decadencia multi-escala; transición de estado sin sobrecarga.

A.4. Mamba: Modelado Selectivo de Espacio de Estado

Los Modelos de Espacio de Estado (SSMs), fuertemente inspirados por teoría de control, originalmente emergieron como alternativas de tiempo continuo a redes neurales discretas. SSMs estructurados tempranos, tales como S4, utilizaban polinomios ortogonales (e.g., matrices HiPPO) para asegurar matemáticamente la memorización de trayectorias históricas largas. Sin embargo, estas arquitecturas tempranas severamente conflictuaron con razonamiento basado en contenido—específicamente tareas requiriendo al modelo ignorar selectivamente ruido o recordar exactamente tokens específicos (e.g., la tarea de inducción head copying). La razón fue fundamental: sus dinámicas de transición eran Invariantes en el Tiempo Lineales (LTI). Mamba directamente resuelve esto introduciendo selectividad a los parámetros fundamentales.

A.4.1. Fundamentación Matemática y el Mecanismo de Selección

Mamba mapea una secuencia de entrada 1D continua $x(t)$ a una secuencia de salida $y(t)$ a través de estados ocultos intermedios $h(t)$. Este sistema está gobernado por un conjunto de ecuaciones diferenciales ordinarias lineales continuas:

$$\begin{aligned} h'(t) &= \mathbf{A}h(t) + \mathbf{B}x(t) \\ y(t) &= \mathbf{C}h(t) \end{aligned}$$

Para operar en tokens discretos de texto o secuencias, estos parámetros continuos deben ser discretizados usando un tamaño de paso Δ. El método de muestreo por orden cero típicamente se emplea para producir matrices discretas $\bar{\mathbf{A}}$ y $\bar{\mathbf{B}}$. En SSMs clásicos, $\mathbf{A}, \mathbf{B}, \mathbf{C}$, y Δ están rígidamente fijados. El avance conceptual central de Mamba es parametrizar $\mathbf{B}, \mathbf{C}$, y Δ como proyecciones lineales de la entrada x_t sí misma:

$$\begin{aligned} s_\Delta &= \text{Lineal}(x_t) \\ \Delta_t &= \text{softplus}(s_\Delta) \\ \mathbf{B}_t &= \text{Lineal}(x_t), \quad \mathbf{C}_t = \text{Lineal}(x_t) \end{aligned}$$

Esta dependencia de entrada transforma el modelo de un sistema invariante en tiempo a un sistema variante en tiempo. Las nuevas reglas de actualización discretizadas se vuelven explícitamente dependientes del token específico en el tiempo t:

$$
\begin{aligned}
\bar{\mathbf{A}}_t &= \exp(\Delta_t \mathbf{A}) \\
\bar{\mathbf{B}}_t &= (\Delta_t \mathbf{A})^{-1}(\exp(\Delta_t \mathbf{A}) - \mathbf{I})\Delta_t \mathbf{B}_t \\
h_t &= \bar{\mathbf{A}}_t h_{t-1} + \bar{\mathbf{B}}_t x_t
\end{aligned}
$$

La similitud matemática a un filtro de Kalman es altamente intencional. Al establecer Δ_t como una función de la entrada, el modelo efectivamente aprende un mecanismo de compuerta. Si un token de entrada es relleno irrelevante, la red puede predecir un Δ_t diminuto (acercándose a cero), causando que $\bar{\mathbf{A}}_t \approx \mathbf{I}$ y $\bar{\mathbf{B}}_t \approx \mathbf{0}$. Esto perfectamente preserva el estado histórico h_{t-1} sin contaminación. Conversamente, encontrando información crítica resulta en un Δ_t grande, completamente refrescando el estado.

A.4.2. Complejidad Computacional y el Escaneo Consciente del Hardware

La introducción de matrices dependientes de entrada rompió la simetría matemática requerida para usar convoluciones FFT (Transformada Rápida de Fourier), las cuales previamente daban a los SSMs su eficiencia de entrenamiento. Para circunvalar esto, los autores engineered un brillante algoritmo de escaneo paralelo consciente del hardware.

Al aprovechar la jerarquía de memoria de la GPU, el algoritmo de escaneo paralelo realiza las actualizaciones secuenciales recurrentes completamente dentro de la SRAM ultra-rápida en-chip. Este proceso circunvala la sobrecarga prohibitiva de ancho de banda de memoria de leer y escribir los estados ocultos de alta dimensión a la HBM más lenta. Consecuentemente, Mamba mantiene complejidad de tiempo de entrenamiento $\mathcal{O}(n \cdot d)$, escalando linealmente con longitud de secuencia mientras retiene beneficios de paralelización de convoluciones.

Durante la inferencia, Mamba opera puramente como una red neuronal recurrente. Logra complejidad de tiempo $\mathcal{O}(1)$ por paso y requiere solo memoria constante $\mathcal{O}(1)$ para almacenar el estado latente h_t. Benchmarks empíricos confirman una mejora de 5x en throughput de inferencia comparado a transformadores de tamaño equivalente, escalando gracefully para manejar secuencias que abarcan hasta un millón de tokens.

A pesar de esto, empujar el modelo a escalas de parámetro de 8B reveló que Mamba puro ligeramente conflictúa comparado a transformadores en tareas requiriendo razonamiento intenso en contexto (e.g., 5-shot MMLU) porque comprimir millones de tokens en un vector fijo fundamentalmente garantiza alguna pérdida de información. Esto ha motivado la creación de arquitecturas híbridas (e.g., Mamba-2-Hybrid) que fusionan 43 % capas Mamba para compresión de largo alcance con 7 % capas de atención para enrutamiento preciso local.

A.4.3. Perfil Arquitectónico: Mamba

Atributo	Especificación
Nomenclatura	Mamba (Modelo Selectivo de Espacio de Estado)
Autores / Año	Gu y Dao (CMU/Princeton) / 2023
Papel / DOI	Mamba: Linear-Time Sequence Modeling with Selective State Spaces / 10.48550/arXiv.2312.00752
Complejidad de Entrenamiento	Tiempo: $\mathcal{O}(n \cdot d)$, Espacio: $\mathcal{O}(n \cdot d)$ (vía escaneo paralelo consciente del hardware)
Complejidad de Inferencia	Tiempo por paso: $\mathcal{O}(1)$, Espacio: $\mathcal{O}(1)$ (vector de estado fijo)
Ventajas	Escalado lineal permite longitudes de secuencia extremas; carga computacional sub-cuadrática; generación autorregresiva drásticamente más rápida.
Desventajas	La compresión de estado intrínsecamente causa degradación menor en recuerdo asociativo denso y tareas de copia exacta comparado a atención.
Características	Matrices discretas dependientes de entrada $(\Delta, \mathbf{B}, \mathbf{C})$; algoritmo de escaneo paralelo fusionado con SRAM; completamente dispensa con autoatención multi-cabeza.

A.5. Transformador ODE: Generación de Secuencias vía Sistemas Dinámicos

Mientras en arquitecturas como RetNet y Mamba enfatizan optimizar interacciones espaciales o recurrencia temporal para maximizar longitud de secuencia, el Transformador ODE aproxima el diseño de redes neurales desde la perspectiva rigurosa de sistemas dinámicos numéricos. Establece una equivalencia matemática profunda entre las capas discretas de un transformador y los métodos de discretización usados para resolver Ecuaciones Diferenciales Ordinarias (EDOs).

A.5.1. Fundamentación Matemática y Refinamiento Runge-Kutta

En una arquitectura de transformador estándar empleando convenciones Pre-Norm, un bloque residual computa la representación de la siguiente capa como $y_{t+1} = y_t + F(y_t, \theta_t)$, donde F encapsula las complejas operaciones de autoatención multi-cabeza y feed-forward, y θ_t representa los parámetros de capa. Desde la perspectiva de sistemas dinámicos, esta formulación es funcionalmente idéntica al método de Euler de primer orden utilizado para aproximar la EDO continua:

$$\frac{dy(t)}{dt} = F(y(t), \theta(t))$$

La discretización de Euler es computacionalmente ineficiente pero notoriamente propensa a acumular errores de truncamiento numérico a través de redes profundas. El Transformador ODE rectifica esta inestabilidad reemplazando la conexión residual simple con solucionadores de Runge-Kutta (RK) de mayor orden, efectivamente permitiendo a la red continuamente refinar sus representaciones dentro de un bloque estructural único.

Por ejemplo, un bloque de Runge-Kutta de segundo orden (RK2) matemáticamente espeja el método de Euler Mejorado:

$$\begin{aligned} y_{t+1} &= y_t + \frac{1}{2}(F_1 + F_2) \\ F_1 &= F(y_t, \theta_t), \quad F_2 = F(y_t + F_1, \theta_t) \end{aligned}$$

La arquitectura escala esto a un bloque Runge-Kutta de cuarto orden (RK4), computando cuatro aproximaciones intermedias precisas para suavizar la trayectoria a través del espacio latente:

$$
\begin{aligned}
y_{t+1} &= y_t + \frac{1}{6}(F_1 + 2F_2 + 2F_3 + F_4) \\
F_1 &= F(y_t, \theta_t) \\
F_2 &= F(y_t + \frac{1}{2}F_1, \theta_t) \\
F_3 &= F(y_t + \frac{1}{2}F_2, \theta_t) \\
F_4 &= F(y_t + F_3, \theta_t)
\end{aligned}
$$

Una propiedad vital de esta arquitectura es el compartimiento de parámetros: el tensor de peso θ_t se reutiliza idénticamente a través de todas las evaluaciones intermedias (F_1 a través de F_4) dentro del bloque. Sin embargo, adherirse estrictamente a los coeficientes numéricos constantes de ecuaciones Runge-Kutta clásicas (e.g., 1/2 o 1/6) dispara desvanecimiento severo de gradientes en modelos profundos. Para preservar estabilidad, el Transformador ODE introduce un mecanismo de compuerta de coeficiente aprendido:

$$
\begin{aligned}
y_{t+1} &= y_t + g \cdot F_1 + (1 - g) \cdot F_2 \\
g &= \text{sigmoid}([\mathbf{F}_1, \mathbf{F}_2]\mathbf{W} + \mathbf{b})
\end{aligned}
$$

A.5.2. Complejidad Computacional y el Compromiso de Precisión

La precisión matemática del Transformador ODE introduce sobrecarga computacional severa. Un bloque RK4 requiere cuatro pases consecutivos hacia adelante de la sub-red F meramente para computar la progresión de una única capa. Mientras el recuento de parámetro actual permanece asombrosamente bajo debido al compartimiento intra-bloque (un bloque RK2 de 6 capas se desempeña equivalentemente a una línea base de 18 capas), la complejidad de tiempo aumenta por un factor constante equivalente al orden RK.

El consumo de memoria también escala agresivamente durante el entrenamiento, como todas las aproximaciones intermedias ($F_1, \ldots, F_n$) deben ser almacenadas para computar gradientes de paso hacia atrás. Benchmarks empíricos revelan una penalidad notable en velocidad de inferencia—decreciendo de 147.1 oraciones por segundo para una red residual línea base a 124.8 oraciones para configuración RK4-block. Sin embargo, este impuesto computacional cede mejoras substanciales en precisión cruda, estableciendo puntuaciones BLEU estado-del-arte (30.77 y 44.11) en tareas de traducción automática a gran escala como WMT'14 English-German y English-French.

A.5.3. Perfil Arquitectónico: Transformador ODE

Atributo	Especificación
Nomenclatura	Transformador ODE
Autores / Año	Li et al. (Northeastern Univ) / 2022
Papel / DOI	ODE Transformer: An Ordinary Differential Equation-Inspired Model for Sequence Generation / 10.18653/v1/2022.acl-long.571
Complejidad de Entrenamiento	Tiempo: $\mathcal{O}(k \cdot n^2)$, Espacio: $\mathcal{O}(k \cdot n^2)$ (donde k es el orden RK)
Complejidad de Inferencia	Tiempo por paso: $\mathcal{O}(k \cdot n)$, Espacio: $\mathcal{O}(n)$
Ventajas	Precisión generativa significativamente más alta vía errores de truncamiento reducidos; altamente eficiente en parámetros debido al compartimiento de peso intra-bloque.
Desventajas	Velocidades de inferencia notablemente más lentas y utilización de memoria más alta; requiere compuerta compleja para evitar desvanecimiento de gradientes.
Características	Solucionadores numéricos de integración Runge-Kutta de orden superior; compuerta de coeficiente dinámico; refinamiento de representación en tiempo continuo.

A.6. Titans: Memorización Neural en Tiempo de Prueba

Como se detalla arriba, modelos recurrentes lineales (RetNet) y modelos de espacio de estado (Mamba) logran eficiencia de inferencia comprimiendo contexto de secuencia en una matriz de estado fijo o vector. Sin embargo, teoría de la información mandata que comprimir una vasta secuencia en una dimensión estática inevitablemente resulta en degradación de datos. La atención estándar evita esto nunca comprimiendo, pero paga el precio en escalado cuadrático. La arquitectura Titans, conceptualizada bajo el marco MIRAS, propone un tercer paradigma radical: "memorización en tiempo de prueba". Bifurca el sistema, utilizando atención para enrutamiento a corto plazo y una red neuronal actualizable por gradiente para actuar como memoria persistente y de largo plazo.

A.6.1. Fundamentación Matemática y la Métrica de Sorpresa

A diferencia de modelos que actualizan un estado latente vía ecuación de recurrencia lineal, Titans mantiene contexto de largo plazo literalmente entrenando los parámetros de un módulo de memoria secundario ($\mathcal{M}$) durante el pase de inferencia hacia adelante. Este enfoque visualiza la adquisición de memoria asociativa como una tarea de meta-aprendizaje en línea.

La memoria neural se basa en una sofisticada regla de actualización de momentum gobernada por una métrica de "sorpresa" S_t. Cuando el modelo encuentra nuevos tokens que contradicen o expanden su memoria asociativa interna, genera un paso de gradiente para permanentemente actualizar los pesos de memoria:

$$\mathcal{M}_t = (1 - \alpha_t)\mathcal{M}_{t-1} + S_t$$
$$S_t = \eta_t S_{t-1} - \theta_t \nabla_\ell(M_{t-1}; x_t)$$

En este sistema, $\nabla_\ell(M_{t-1}; x_t)$ representa la sorpresa momentánea—calculada como el gradiente de una función de pérdida asociativa ℓ evaluada en la entrada actual x_t. Esta formulación cercanamente asemeja descenso de gradiente con momentum, donde S_t rastrea la sorpresa acumulada a través del tiempo.

Los coeficientes η_t y θ_t son parámetros críticos de decadencia dependiente de datos y tasa de aprendizaje. Porque son funciones de la entrada, el modelo dinámicamente dicta la asimilación de memoria. Si la secuencia transiciona a un contexto semántico completamente nuevo, el modelo puede establecer $\eta_t \to 0$, forzando a la memoria a ignorar momentum acumulado y rápidamente adaptarse al nuevo paradigma. Alternativamente, establecer $\eta_t \to 1$ completamente incorpora la sorpresa histórica en la actualización de peso.

La salida del módulo de memoria neuronal profunda se recupera consultando el módulo actualizado con el token actual, creando un mecanismo de compuerta elemento a elemento con el estado oculto estándar:

$$o_t = y_t \otimes \mathcal{M}_t^*(y_t)$$

A.6.2. Complejidad Computacional e Contexto Infinito

Ejecutar actualizaciones de peso basadas en gradiente durante un pase de inferencia autorregresiva inicialmente aparece computacionalmente prohibitivo. Sin embargo, Titans están engineered para ser altamente optimizados. Porque el módulo de memoria neural está limpianamente separado de la ventana de atención a corto plazo, la operación de atención estándar local mantiene una complejidad altamente manejable $\mathcal{O}(c^2)$ en fragmentos de contexto localizados.

Simultáneamente, la actualización de memoria neuronal de largo plazo logra complejidad lineal $\mathcal{O}(n)$ relativa a la longitud total de secuencia. La ventaja arquitectónica más profunda es que la memoria está estructuralmente incrustada dentro de los parámetros físicos del módulo en lugar de externalizada como un caché KV expandible. Consecuentemente, requiere una huella de memoria fija $\mathcal{O}(1)$ en tiempo de prueba, completamente circunvalando los requisitos exhaustivos de HBM asociados con decodificación de secuencias masivas.

A.6.3. Perfil Arquitectónico: Titans

Atributo	**Especificación**
Nomenclatura	Titans (Arquitectura de Memoria Neural)
Autores / Año	Behrouz et al. / 2025
Papel / DOI	Titans: Testing-Time Training with Augmented Attention / 10.48550/arXiv.2501.00663
Complejidad de Entrenamiento	Tiempo: $\mathcal{O}(n^2)$ (atención local) + $\mathcal{O}(n)$ (memoria), Espacio: $\mathcal{O}(1)$ (sin caché KV)
Complejidad de Inferencia	Tiempo por paso: $\mathcal{O}(c^2 + d \cdot k)$ (donde c es el tamaño de ventana local, d es dimensión, k es rango de memoria)
Ventajas	Contexto efectivamente infinito sin expansión de caché KV; memoria asociativa persistente; comportamiento adaptativo dinámico a cambios de distribución.
Desventajas	Mayor complejidad de implementación; requiere cómputo diferenciable durante inferencia; tuning de parámetros de control de sorpresa es crítico.
Características	Actualización de parámetros en tiempo de ensayo; métrica de sorpresa dependiente de datos; fusión de atención local con memoria persistente; framwork de meta-aprendizaje en línea.

A.7. Síntesis Comparativa: Posicionamiento Arquitectónico

La siguiente tabla sintetiza los atributos clave a través de las seis arquitecturas, ayudando a entender sus perfil de compromiso:

Arquitectura	Comp. Entrenar	Comp. Inferir	KV Cache	Contexto	Desempeño	
Atención Estándar	$\mathcal{O}(n^2)$	$\mathcal{O}(n)$	Requerido	Limitado	Excelente	
Atención Sigmoide	$\mathcal{O}(n^2)$	$\mathcal{O}(n)$	Requerido	Limitado	Muy Bueno	Re
RetNet	$\mathcal{O}(n)$–$\mathcal{O}(n^2)$	$\mathcal{O}(1)$	No	Muy Largo	Bueno	
Mamba	$\mathcal{O}(n)$	$\mathcal{O}(1)$	No	Muy Largo	Muy Bueno	
Transformador ODE	$\mathcal{O}(k \cdot n^2)$	$\mathcal{O}(k \cdot n)$	Requerido	Limitado	Excelente	
Titans	$\mathcal{O}(n^2)$	$\mathcal{O}(c^2)$	No	Infinito	Excelente	

A.7.1. Guía de Selección

- **Para Desempeño Máximo**: Atención Estándar o Transformador ODE. La atención estándar mantiene la máxima expresividad; Transformador ODE proporciona refinamiento intra-bloque a mayor costo computacional.
- **Para Inferencia Eficiente en Hardware**: RetNet o Mamba. RetNet ofrece cambio paradigma limpio entre entrenamiento y inferencia; Mamba proporciona mejor escalado lineal en entrenamiento sin compromiso de expresividad.
- **Para Contexto Extremadamente Largo**: Mamba, Titans, o RetNet. Mamba sin caché KV, Titans con memoria persistente, RetNet con estado comprimido permiten el mayor contexto sin agotamiento de memoria.
- **Para Despliegue en Dispositivos Limitados**: Mamba (bajo consumo de memoria, $\mathcal{O}(1)$ inferencia) o Transformador ODE si desempeño es crítico.
- **Para Tareas Que Requieren Comportamiento Adaptativo**: Titans. Su capacidad de actualizar memoria en tiempo de prueba responde a distribuciones de entrada dinámicas.

A. Tipos de Atención Dispersa en Transformadores: Revisión Exhaustiva

A.1. Resumen Ejecutivo

La autoatención estándar en Transformadores calcula interacciones por pares para todos los tokens, resultando en complejidad computacional y de memoria $\mathcal{O}(n^2)$. Esto se vuelve prohibitivo para secuencias largas. Los mecanismos de atención dispersa abordan esto restringiendo el conjunto de interacciones de tokens, explotando la observación empírica de que la mayoría de los pesos de atención aprendidos están cerca de cero. Este apéndice revisa siete tipos prominentes de bloques de atención dispersa—desde métodos fundamentales como Sparse Transformer hasta los enfoques más recientes como FASA y NSA—proporcionando para cada uno una descripción, formulación matemática, ventajas/desventajas e implementación en PyTorch.

A.2. Sparse Transformer (Child et al., 2019)

A.2.1. Descripción

El Sparse Transformer fue uno de los primeros trabajos en introducir **atención dispersa factorizada**, reduciendo la complejidad de $\mathcal{O}(n^2)$ a $\mathcal{O}(n\sqrt{n})$. En lugar de calcular una matriz de atención completa, factoriza el patrón de atención en dos patrones dispersos complementarios—**atención con zancada** y **atención fija**—cada uno atendiendo a $\mathcal{O}(\sqrt{n})$ posiciones. Al componer estos sobre dos cabezas de atención, cada posición puede atender a cada otra posición a través de un camino de longitud como máximo $p + 1$, donde p es el número de cabezas factorizado.

El método fue aplicado a imágenes, audio y texto, estableciendo resultados de vanguardia en puntos de referencia de modelado de densidad Enwik8, CIFAR-10 e ImageNet-64.

A.2.2. Formulación Matemática

Sea n la longitud de secuencia y $l = \lfloor \sqrt{n} \rfloor$ la zancada. Dos cabezas de atención factorizado se definen:

Cabeza de zancada — cada posición i atiende a posiciones en $\{j : (i - j) \text{ mód } l = 0\}$, es decir, cada posición previa de l-ésima:

$$A_i^{(\text{zancada})} = \{j : j \leq i,\ (i - j) \text{ mód } l = 0\}$$

Cabeza fija — cada posición i atiende a una ventana local más posiciones de columna fija:

$$A_i^{(\text{fija})} = \{j : \lfloor j/l \rfloor = \lfloor i/l \rfloor\} \cup \{j : j \text{ mód } l \in \{l-c, \ldots, l-1\}\}$$

donde c es un hiperparámetro para el número de columnas de resumen. La salida de atención general por cabeza sigue atención de producto punto escalado restringida a cada conjunto A_i:

$$\text{Atención}(Q, K, V)_i = \text{softmax}\left(\frac{q_i \cdot K_{A_i}^\top}{\sqrt{d_k}}\right) V_{A_i}$$

A.2.3. Ventajas y Desventajas

Ventajas	**Desventajas**
Reduce complejidad de $\mathcal{O}(n^2)$ a $\mathcal{O}(n\sqrt{n})$	Los patrones fijos pueden perder dependencias largo-rango importantes
Probado en imágenes, audio, texto	Requiere kernels CUDA personalizados para eficiencia
Habilita secuencias de 10K+ tokens con cientos de capas	El patrón de zancada es agnóstico de datos

A.3. Longformer (Beltagy et al., 2020)

A.3.1. Descripción

Longformer introduce un mecanismo de atención que **escala linealmente** con la longitud de secuencia combinando tres patrones: (1) **atención de ventana deslizante**, donde cada token atiende a una ventana fija de w vecinos; (2) **atención de ventana deslizante dilatada**, que introduce espacios de tamaño d para expandir el campo receptivo; y (3) **atención global** en tokens específicos de la tarea (por ejemplo, `[CLS]`) que atienden y son atendidos por todos los tokens. Con L capas y tamaño de ventana w, el campo receptivo de capa superior es $L \times w$, cubriendo la secuencia completa eficientemente.

A.3.2. Formulación Matemática

Para un token en posición i con una ventana deslizante de tamaño w:

$$A_i^{(\text{desliza})} = \{j : |i - j| \leq w/2\}$$

La variante dilatada con dilatación d:

$$A_i^{(\text{dilatada})} = \{j : |i-j| \leq w/2 \cdot (d+1),\ (i-j) \text{ mód } (d+1) = 0\}$$

Para tokens globales en conjunto $\mathcal{G}$:

$$A_i^{(\text{global})} = \{1, \ldots, n\} \quad \text{si } i \in \mathcal{G}, \qquad A_i^{(\text{desliza})} \cup \mathcal{G} \quad \text{si no}$$

La complejidad general es $\mathcal{O}(n \cdot w)$, que es lineal en n para w fijo.

A.3.3. Ventajas y Desventajas

Ventajas	**Desventajas**
Complejidad lineal $\mathcal{O}(n \cdot w)$	El tamaño de ventana limita contexto local por capa
Maneja secuencias de 4096+ tokens	Los tokens globales deben ser elegidos específicamente por tarea
Reemplazo intercambiable para atención estándar	Los patrones dilatados pueden perder detalle local fino
Flexible: diferente w por capa	Requiere kernels CUDA dispersos personalizados para eficiencia

A.4. BigBird (Zaheer et al., 2020)

A.4.1. Descripción

BigBird combina tres patrones de atención dispersa—**aleatorio**, **ventana deslizante (local)** y **tokens globales**—para lograr **complejidad lineal** $\mathcal{O}(n)$ mientras preserva las propiedades de completitud de Turing y aproximación universal de atención completa. La contribución teórica prueba que $\mathcal{O}(1)$ tokens globales son suficientes para mantener estas propiedades. BigBird maneja secuencias 8× más largas que BERT en el mismo hardware y logra vanguardia en tareas de preguntas-respuestas, resumen y genómica.

A.4.2. Formulación Matemática

Para cada token i, el conjunto atendido es:

$$A_i = A_i^{(\text{aleatorio})} \cup A_i^{(\text{ventana})} \cup A_i^{(\text{global})}$$

donde:

- **Aleatorio**: r posiciones elegidas aleatoriamente por token
- **Ventana**: $A_i^{(\text{ventana})} = \{j : |i-j| \leq w/2\}$ para tamaño de ventana w
- **Global**: un conjunto de g tokens que atienden a/desde todas las posiciones

La máscara general M es:

$$M_{ij} = \mathbf{1}\left[j \in A_i^{(\text{aleatorio})} \cup A_i^{(\text{ventana})} \cup A_i^{(\text{global})}\right]$$

La atención se calcula como:

$$\text{Atención}(Q, K, V)_i = \text{softmax}\left(\frac{q_i K_{A_i}^\top}{\sqrt{d_k}}\right) V_{A_i}$$

A.4.3. Ventajas y Desventajas

Ventajas	Desventajas
Aproximador universal comprobado y Turing completo	Los patrones aleatorios introducen no-determinismo
Complejidad lineal $\mathcal{O}(n)$	La granularidad de bloque puede desperdiciar cómputo
Maneja 8× secuencias más largas que BERT	Requiere sintonización cuidadosa de r, w, g
Fuerte en P+R, resumen, genómica	Menos efectivo en tareas que necesitan atención global densa

A.5. SparseK Attention (Lou et al., 2024)

A.5.1. Descripción

SparseK Attention introduce un **operador top-k diferenciable** para atención dispersa que habilita optimización basada en gradientes. Una red de puntuación aprendida evalúa la importancia de cada par clave-valor, y el operador SparseK selecciona un número constante k de pares KV por consulta. Esto produce **complejidad temporal lineal** durante entrenamiento y **huella de memoria constante** durante generación autorregresiva. Se integra sin inconvenientes en LLMs entrenados con sintonización mínima.

A.5.2. Formulación Matemática

Para cada consulta q, una red de puntuación produce puntuaciones de importancia $u \in \mathbb{R}^n$ para todos los pares KV. El **operador SparseK** calcula un umbral $\tau(u)$ tal que la suma de puntuaciones normalizadas sea igual a k:

$$\text{SparseK}(u, k)_j = \text{máx}(u_j - \tau(u), 0)$$

donde τ se elige tal que $\sum_j \text{máx}(u_j - \tau, 0) = k$. Esta es una relajación diferenciable de top-k. La atención es entonces:

$$m_j = \text{SparseK}(u, k)_j, \quad \text{Atención}(q, K, V) = \text{softmax}\left(\frac{qK_{\text{sel}}^\top}{\sqrt{d_k}}\right) V_{\text{sel}}$$

donde $K_{\text{sel}}, V_{\text{sel}}$ contienen solo las entradas top-k.

A.5.3. Ventajas y Desventajas

Ventajas	Desventajas
Diferenciable, soporta entrenamiento de extremo a extremo	La red de puntuación añade sobrecarga
Tiempo lineal, memoria constante en generación	Requiere sintonización fina cuando se aplica a modelos pre-entrenados
Integración sin inconvenientes en LLMs existentes	Los k fijo pueden no ser óptimos para todas las capas/cabezas
Supera métodos de atención dispersa previos	La selección top-k no está aligned con hardware

A.6. NSA — Native Sparse Attention (Yuan et al., 2025)

A.6.1. Descripción

NSA, desarrollado por DeepSeek, es un mecanismo de atención dispersa **entrenable nativamente** que integra optimizaciones alineadas con hardware con una **estrategia dispersa jerárquica dinámica**. A diferencia de métodos solo para inferencia, NSA soporta entrenamiento de extremo a extremo, reduciendo cómputo de preentrenamiento sin sacrificar desempeño. Procesa claves y valores a través de tres ramas de atención paralelas:

1. **Atención comprimida**: tokens de grano grueso mediante un MLP aprendible que agrega bloques
2. **Atención seleccionada**: selección de bloque top-n de grano fino basada en puntuaciones comprimidas
3. **Ventana deslizante**: contexto local de los w tokens más recientes

Las salidas se combinan a través de un mecanismo de compuerta aprendido. NSA logra aceleración de descodificación de hasta 11.6× y aceleración forward de 9× en secuencias de longitud 64k.

A.6.2. Formulación Matemática

La atención general reemplaza $(\mathbf{k}_{1:t}, \mathbf{v}_{1:t})$ completa con representaciones compactas mediante tres estrategias:

$$o_t^* = \sum_{c \in \{\text{cmp,slc,win}\}} g_t^c \cdot \text{Atención}(q_t, \tilde{K}_t^c, \tilde{V}_t^c)$$

donde $g_t^c \in [0, 1]$ son puntuaciones de compuerta aprendidas de un MLP + sigmoid.
Compresión — Un MLP aprendible φ mapea bloques de l claves con zancada d:

$$\tilde{K}_t^{\text{cmp}} = (\varphi(k_{id+1:id+l}))_{1 \le i \le \lfloor (t-l)/d \rfloor}$$

Selección — Importancia de bloque desde puntuaciones de atención comprimida:

$$p_t^{\text{cmp}} = \text{softmax}(q_t^\top \tilde{K}_t^{\text{cmp}}), \quad I_t = \{i : \text{rango}(p_t^{\text{slc}'}[i]) \le n\}$$

Ventana deslizante: $\tilde{K}_t^{\text{win}} = k_{t-w:t}$.
Tokens totales atendidos por consulta: $N_t = \lfloor t/d \rfloor + n \cdot l' + w \ll t$.

A.6.3. Ventajas y Desventajas

Ventajas	Desventajas
Entrenable de extremo a extremo nativamente	Requiere kernels Triton personalizados
Diseño alineado con hardware (utilización Tensor Core)	Arquitectura compleja multi-rama
11.6× aceleración descodificación, 9× forward a 64k	Necesita backbone GQA/MQA para mejores resultados
Supera atención completa en muchos benchmarks	El MLP de compresión añade parámetros
Diseño jerárquico captura patrones locales y globales	Mayor complejidad de implementación vs. métodos más simples

A.7. SpargeAttn (Zhang et al., 2025)

A.7.1. Descripción

SpargeAttn es un método de atención dispersa **universal sin entrenamiento** que acelera modelos diversos—generación de lenguaje, imagen y video—usando un **filtro en línea de dos etapas** construido sobre FlashAttention. La primera etapa predice rápidamente qué bloques de la matriz de atención contendrán valores cercanos a cero y salta las multiplicaciones $Q_i K_j^\top$ correspondientes. La segunda etapa aplica un **filtro consciente de softmax en línea** sin sobrecarga adicional para eliminar más cómputos $\tilde{P}_{ij} V_j$ innecesarios. SpargeAttn logra aceleración de 2.5–5× mientras preserva métricas de extremo a extremo.

A.7.2. Formulación Matemática

SpargeAttn opera en nivel de bloque. Para bloque de consulta Q_i y bloque de clave K_j:

Etapa 1 — Predicción Dispersa: Estimar si Atención(Q_i, K_j) será negligible. Esto se hace computando un proxy de bajo costo (por ejemplo, usando representaciones comprimidas o auto-similitud de token):

$$\hat{s}_{ij} = f_{\text{pred}}(Q_i, K_j) \quad \Rightarrow \quad \text{saltar si } \hat{s}_{ij} < \epsilon_1$$

Etapa 2 — Filtro Consciente de Softmax: Después de computar $\tilde{P}_{ij} = \text{softmax}(Q_i K_j^\top / \sqrt{d_k})$, verificar si la contribución del bloque es negligible relativa al máximo de softmax en línea corrida:

$$\text{saltar } \tilde{P}_{ij} V_j \quad \text{si } \ \text{máx}(\tilde{P}_{ij}) \ll e^{m_{\text{old}} - m_{\text{new}}}$$

donde $m_{\text{old}}, m_{\text{new}}$ son máximos de softmax en línea corrida.

A.7.3. Ventajas y Desventajas

Ventajas	Desventajas
Universal: funciona en LLMs, diffusion de imagen y video	La aceleración depende de sparsidad inherente
Sin entrenamiento, plug-and-play	La granularidad de bloque puede perder patrones finos
2.5–5× más rápido que atención densa/dispersa existente	El filtro de dos etapas añade algo de sobrecarga constante
Mejora desempeño de LLM con contexto largo	Requiere integración kernel Triton/CUDA
Compatible con cuantificación	Sintonización de umbral ϵ necesaria por familia de modelo

A.8. FASA — Frequency-Aware Sparse Attention (Wang et al., 2026)

A.8.1. Descripción

FASA es un marco de atención dispersa **sin entrenamiento** que aprovecha una insight novedosa sobre RoPE (Rotary Position Embeddings): solo un pequeño subconjunto de **fragmentos de frecuencia** (FCs) en cada cabeza de atención contribuye a conciencia contextual, mientras que la mayoría codifica patrones posicionales. Estos "FCs dominantes" son **dispersos** (menos del 1 % de todos los FCs), **universales** a través de escalas de modelo, y **agnósticos de tarea**. FASA opera en dos etapas: (1) **Predicción de Importancia de Token (TIP)** usa FCs dominantes para estimar barato qué tokens importan, y (2) **Cómputo de Atención Enfocada (FAC)** realiza atención de precisión completa solo en el subconjunto seleccionado.

FASA logra prácticamente 100 % del desempeño de KV completo en LongBench mientras mantiene solo 256 tokens, y entrega aceleración de 2.56× con uso de caché de 18.9 %.

A.8.2. Formulación Matemática

Descomposición de Fragmento de Frecuencia bajo RoPE. Cada vector de dimensión d se divide en $d/2$ fragmentos 2D $\mathbf{v}^{[i]} = (v_{2i}, v_{2i+1})^\top$. La rotación RoPE es diagonal por bloques:

$$\mathbf{R}_{\Delta t} = \bigoplus_{i=1}^{d/2} \mathbf{R}_{\Delta t, \theta_i}, \quad \theta_i = B^{-2(i-1)/d}$$

Acuerdo Contextual (CA) mide la dominancia de FC i en cabeza (l, h):

$$\mathrm{CA}_{\mathcal{K}}^{l,h,i}(\mathbf{q}_t, \mathbf{K}_{1:t}) = \frac{|\text{TopK-I}(\boldsymbol{\alpha}_{l,h}, \mathcal{K}) \cap \text{TopK-I}(\boldsymbol{\alpha}_{l,h}^{(i)}, \mathcal{K})|}{\mathcal{K}}$$

Etapa TIP — Puntuación de importancia en línea usando solo FCs dominantes $\mathcal{I}_{\text{dom}}$:

$$\mathbf{S}_t^{l,h} = \sum_{i \in \mathcal{I}_{\text{dom}}^{l,h}} \boldsymbol{\alpha}^{l,h,i}(\mathbf{q}_t, \mathbf{K}_{1:t}), \quad \mathcal{T}_t = \text{TopK-I}(\mathbf{S}_t^{l,h}, N_{\text{fac}})$$

Etapa FAC — Atención de precisión completa en tokens seleccionados:

$$\hat{\boldsymbol{\alpha}}_{\text{FAC}}^{l,h} = \text{softmax}\left(\frac{\mathbf{q}_t \mathbf{K}_{\mathcal{T}_t}^\top}{\sqrt{d}}\right), \quad \mathbf{O}_t^{l,h} = \hat{\boldsymbol{\alpha}}_{\text{FAC}}^{l,h} \mathbf{V}_{\mathcal{T}_t}$$

Complejidad general: $\mathcal{O}(2t \cdot N_{\text{tip}} + 2N_{\text{fac}} \cdot d)$ vs. $\mathcal{O}(2td)$ para atención completa. Aceleración $\approx d/N_{\text{tip}}$ cuando $N_{\text{fac}} \ll t$.

A.8.3. Ventajas y Desventajas

Ventajas	Desventajas
Sin entrenamiento, plug-and-play	Requiere modelos basados en RoPE (extendible a ALiBi/MLA)
Precisión casi-oracle con ≤ 256 tokens	Paso de calibración offline necesario (aunque una sola vez)
Dos variantes: FASA-M (memoria), FASA-C (cómputo)	El análisis de dominancia de FC añade complejidad de implementación
Hasta 8× compresión caché KV y aceleración 2.56×	Desempeño depende de calidad de identificación de FC dominante
Ortogonal a otros métodos de compresión KV	Limitado a optimización de fase de descodificación

A.9. Tabla Comparativa Resumen

Método	Año	Complejidad	Huella Mem.	Entr.	S.Entr.	Innovación Clave
Sparse Transformer	2019	$\mathcal{O}(n\sqrt{n})$	$\mathcal{O}(n\sqrt{n})$	Sí	No	Patrones zancada+fijo factorizado
Longformer	2020	$\mathcal{O}(n \cdot w)$	$\mathcal{O}(n \cdot w)$	Sí	No	Ventana + dilatada + global
BigBird	2020	$\mathcal{O}(n)$	$\mathcal{O}(n)$	Sí	No	Aleatorio + ventana + global
SparseK Attn.	2024	$\mathcal{O}(n)$ / $\mathcal{O}(k)$	$\mathcal{O}(k)$	Sí	No	Op. top-k diferenciable
NSA	2025	$\mathcal{O}(t/d + nl' + w)$	$\mathcal{O}(t/d + nl' + w)$	Sí	No	Comprime+selecciona+ventana
SpargeAttn	2025	$\mathcal{O}(n^2 \cdot s)$	$\mathcal{O}(n)$	No	Sí	Filtro dos etapas
FASA	2026	$\mathcal{O}(t \cdot N_t + N_f \cdot d)$	$\mathcal{O}(N_f \cdot d)$	No	Sí	Sparsidad freq. en RoPE

Notas: n = longitud de secuencia; w = tamaño de ventana; k = pares KV seleccionados; t = longitud de contexto; d = dimensión de cabeza; N_{tip} = número de FCs dominantes; N_{fac} = número de tokens seleccionados para atención enfocada; l' = tamaño de bloque de selección; s = fracción de bloques no dispersos.

A.10. Guía de Selección y Recomendaciones

A.10.1. Para Máximo Desempeño

Los métodos **Sparse Transformer**, **BigBird** y **NSA** ofrecen compensaciones equilibradas entre complejidad y exactitud. El Sparse Transformer estándar es el más probado empíricamente; NSA es el más moderno y eficiente en hardware.

A.10.2. Para Inferencia Eficiente

Los métodos sin entrenamiento **FASA** y **SpargeAttn** son plug-and-play ideales, requiriendo zero modificación de modelos pre-entrenados. FASA sobresale con restricciones de caché KV severas; SpargeAttn es universal a través de arquitecturas.

A.10.3. Para Contextos Extremadamente Largos

SparseK, NSA y BigBird soportan contextos de decenas de miles de tokens. NSA es el más eficiente computacionalmente; BigBird es el más probado teóricamente.

A.10.4. Para Dispositivos Limitados en Memoria

SparseK (memoria constante durante inferencia) y FASA (compresión de 8× caché) son óptimos. Longformer es también práctico para memoria limitada con ventana de tamaño controlable.

A.10.5. Para Modelos Existentes Sin Reentrenamiento

FASA y SpargeAttn son el único camino: ambos operan sin entrenamiento. FASA requiere calibración offline; SpargeAttn es completamente dinámico.

A. Bloques de Atención con Compuerta en Transformadores: Una Revisión Completa

A.1. Introducción y Motivación

El mecanismo de atención softmax estándar en Transformadores, aunque poderoso, sufre de complejidad cuadrática en la longitud de la secuencia y carece de mecanismos explícitos para gestión de memoria. Entre 2023 y 2025, ha surgido una familia de arquitecturas de **atención con compuerta** para abordar estas limitaciones mediante la incorporación de compuertas dependientes de datos en el cálculo de atención. Estos mecanismos se inspiran en las compuertas de olvido en redes neuronales recurrentes (LSTM, GRU) y las aplican a atención lineal, modelos de espacio de estado o incluso atención softmax misma.

Esta sección cubre siete arquitecturas clave, proporcionando para cada una: descripción, formulación matemática, ventajas y limitaciones, y un pseudocódigo de algoritmo estilo paper.

A.2. 1. Atención Lineal con Compuerta (GLA)

A.2.1. Descripción

Atención Lineal con Compuerta (GLA, Yang et al., 2023) aumenta la atención lineal vanilla con un mecanismo de compuerta dependiente de datos. La atención lineal reformula la atención estándar como una recurrencia con estado oculto valuado en matrices $\mathbf{S}_t = \mathbf{S}_{t-1} + \mathbf{v}_t\mathbf{k}_t^\top$, permitiendo inferencia con memoria constante. Sin embargo, esta acumulación puramente aditiva conduce a "sobrecarga de memoria"—asociaciones antiguas nunca pueden ser borradas. GLA introduce una matriz de compuerta diagonal $\mathbf{G}_t$ para desintegrar selectivamente información histórica, reduciendo significativamente la brecha de rendimiento con atención softmax.

El algoritmo *FLASHLINEARATTENTION* logra complejidad de entrenamiento sub-cuadrática mediante una estrategia de bloques paralela que maximiza la utilización de núcleos tensoriales.

A.2.2. Formulación Matemática

La forma recurrente de GLA es:

$$\mathbf{S}_t = \mathbf{G}_t \odot \mathbf{S}_{t-1} + \mathbf{v}_t\mathbf{k}_t^\top, \quad \mathbf{o}_t = \mathbf{S}_t\mathbf{q}_t \tag{1}$$

donde $\mathbf{G}_t \in \mathbb{R}^{d\times d}$ es parametrizado con estructura de producto exterior $\mathbf{G}_t = \mathbf{1}\boldsymbol{\alpha}_t^\top$, y $\boldsymbol{\alpha}_t$ se calcula vía proyección de bajo rango seguida de activación logsigmoide y normalización:

$$\boldsymbol{\alpha}_t = \frac{\log \sigma(\mathbf{W}_{gk}\mathbf{x}_t)}{c} \tag{2}$$

donde c es un normalizador (típicamente 16). En forma paralela por bloques:

$$\mathbf{O}_{[t]} = \overleftarrow{\mathbf{Q}}_{[t]}\mathbf{S}_{[t]}^{\top} + \left(\mathbf{Q}_{[t]}\mathbf{K}_{[t]}^{\top} \odot \mathbf{\Gamma}_{[t]}\right)\mathbf{V}_{[t]} \tag{3}$$

donde $\mathbf{\Gamma}$ es la máscara causal consciente de descomposición.

A.2.3. Pseudocódigo

Algorithm 1 Atención Lineal con Compuerta (GLA) - Forma Recurrente

Require: Entrada secuencial $x_{1:L}$, proyecciones aprend. W_q, W_k, W_v, W_{gk}
Ensure: Salida atendida $\mathbf{O}$

```
S ← 0_{d×d}, O ← []
for t = 1 to L do
    q_t ← W_q(x_t), k_t ← W_k(x_t), v_t ← W_v(x_t)
    α_t ← log σ(W_gk(x_t))/16
    α_t ← exp(α_t)        ▷ descomposición por elemento
    S ← S · α_t + v_t k_t^⊤        ▷ actualización de estado
    o_t ← S q_t        ▷ lectura de estado
    Agregar o_t a O
end for
```

A.2.4. Ventajas y Limitaciones

Ventajas	**Limitaciones**
Entrenamiento sub-cuadrático vía paralelismo por bloques	Desempeño inferior a atención softmax en tareas de recuperación
Memoria constante $\mathcal{O}(d^2)$ en inferencia	Estructura de compuerta limita selectividad por valor-clave individual
Buena generalización de longitud (2K $\rightarrow$ 20K+)	Aún existe una brecha vs. Transformadores en tareas de contexto largo
Mayor rendimiento que Mamba a escala similar	Requiere kernels CUDA personalizados para velocidad óptima

A.3. 2. DeltaNet

A.3.1. Descripción

DeltaNet aplica la regla de aprendizaje clásica **delta**—un principio de corrección de errores—a atención lineal. En lugar de meramente acumular productos externos clave-valor, DeltaNet actualiza su estado computando la diferencia entre el valor predicho (recuperado de memoria vía la clave actual) y el valor objetivo, luego corrigiendo en consecuencia. Esto es matemáticamente equivalente a ejecutar un paso de descenso de gradiente estocástico en una pérdida MSE *online* en cada paso temporal, conectando DeltaNet al paradigma de entrenamiento en tiempo de prueba (TTT).

DeltaNet demuestra desempeño perfecto en recuperación asociativa multi-consulta (MQAR), un benchmark sintético crítico, y supera modelos lineales con compuerta en tareas de recuperación en contexto.

A.3.2. Formulación Matemática

La actualización de estado DeltaNet (regla delta) es:

$$\mathbf{S}_t = \mathbf{S}_{t-1}(\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) + \beta_t \mathbf{v}_t \mathbf{k}_t^\top \tag{4}$$

donde $\beta_t \in (0, 1)$ es la "fortaleza de escritura" dependiente de datos (sigmoide de proyección lineal), y las claves están normalizadas L2. Esto puede descomponerse como operación borrar-escribir:

$$\mathbf{v}_t^{\text{nueva}} = (1 - \beta_t)\mathbf{v}_t^{\text{vieja}} + \beta_t \mathbf{v}_t, \quad \mathbf{S}_t = \mathbf{S}_{t-1} - \mathbf{v}_t^{\text{vieja}} \mathbf{k}_t^\top + \mathbf{v}_t^{\text{nueva}} \mathbf{k}_t^\top \tag{5}$$

Conexión con aprendizaje online: DeltaNet minimiza $\mathcal{L}_t(\mathbf{S}) = \frac{1}{2}\|\mathbf{S}\mathbf{k}_t - \mathbf{v}_t\|^2$ vía descenso de gradiente con tasa de aprendizaje β_t.

A.3.3. Pseudocódigo

Algorithm 2 DeltaNet - Regla Delta para Atención Lineal

Require: Entrada $x_{1:L}$, normalizador L2
Ensure: Salida $\mathbf{O}$

```
S ← 0_{d×d}, O ← []
for t = 1 to L do
    q_t ← NORM(W_q(x_t)), k_t ← NORM(W_k(x_t))
    v_t ← W_v(x_t)
    β_t ← σ(W_β(x_t))    ▷ fortaleza de escritura
    S ← S(I − β_t k_t k_t^⊤) + β_t v_t k_t^⊤
    o_t ← S q_t
    Agregar o_t a O
end for
```

A.3.4. Ventajas y Limitaciones

Ventajas	**Limitaciones**
Recuperación asociativa superior en contexto (MQAR perfecto)	Sin olvido global → abarrotamiento de memoria en secuencias largas reales
Teóricamente principiado (optimización MSE online)	Ligeramente más lento que Mamba2 por transiciones más ricas
Capacidades fuertes de seguimiento de estado	Desempeño modesto en recuperación en contexto de mundo real
Entrenamiento paralelo eficiente vía representación WY	Requiere claves normalizadas L2 para estabilidad

A.4. 3. Gated DeltaNet

A.4.1. Descripción

Gated DeltaNet (Yang et al., 2024) es una síntesis natural del mecanismo de compuerta de Mamba2/GLA y la regla delta de DeltaNet. La perspectiva clave es que estos dos mecanismos son **complementarios**: la compuerta permite borrado rápido de memoria (estableciendo $\alpha_t \to 0$ limpia el estado), mientras que la regla delta permite actualizaciones dirigidas por clave-valor (estableciendo $\alpha_t \to 1$ recupera regla delta pura).

Publicado en ICLR 2025, Gated DeltaNet supera consistentemente tanto a Mamba2 como a DeltaNet en modelado de lenguaje, razonamiento de sentido común, recuperación en contexto, extrapolación de longitud y comprensión de contexto largo. Ha sido integrado en modelos Qwen3-Next y Qwen3.5 de Alibaba.

A.4.2. Formulación Matemática

La actualización de estado de **regla delta con compuerta** es:

$$\mathbf{S}_t = \mathbf{S}_{t-1}\left(\alpha_t(\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top)\right) + \beta_t \mathbf{v}_t \mathbf{k}_t^\top \tag{6}$$

donde $\alpha_t \in (0,1)$ es la compuerta de descomposición y $\beta_t \in (0,1)$ es la fortaleza de escritura. Desde la perspectiva de aprendizaje online, esto corresponde a:

$$\|\mathbf{S}_t - \alpha_t \mathbf{S}_{t-1}\|_F^2 - 2\langle \mathbf{S}_t \mathbf{k}_t, \beta_t(\mathbf{v}_t - \alpha_t \mathbf{S}_{t-1}\mathbf{k}_t)\rangle \tag{7}$$

Esto introduce decaimiento de peso adaptativo α_t en la actualización estilo SGD, análogo a decoupled weight decay en optimización de aprendizaje profundo.

A.4.3. Pseudocódigo

Algorithm 3 Gated DeltaNet - Regla Delta Compuertada

Require: Entrada $x_{1:L}$, normalizador L2
Ensure: Salida $\mathbf{O}$

```
S ← 0_{d×d}, O ← []
for t = 1 to L do
    q_t ← NORM(W_q(x_t)), k_t ← NORM(W_k(x_t))
    v_t ← W_v(x_t)
    α_t ← σ(W_α(x_t))    ▷ compuerta de descomposición
    β_t ← σ(W_β(x_t))    ▷ fortaleza de escritura
    S ← S · α_t · (I − β_t k_t k_t^⊤) + β_t v_t k_t^⊤
    o_t ← S q_t
    Agregar o_t a O
end for
```

A.4.4. Ventajas y Limitaciones

Ventajas	**Limitaciones**
Mejor entre modelos recurrentes lineales puros	Rendimiento ligeramente menor que Mamba2 (matrices de transición más ricas)
Complementariedad compuerta + regla delta para gestión de memoria	Requiere kernels triton/CUDA personalizados para producción
Entrenamiento paralelo eficiente	Dos compuertas (α, β) aumentan sensibilidad a hiperparámetros
Variantes híbridas (con atención de ventana) superan Transformadores	Acotado por tamaño fijo de estado para recuperación exacta

A.5. 4. RetNet (Red Retentiva)

A.5.1. Descripción

RetNet, propuesto por Sun et al. (2023) en Microsoft Research, introduce un **mecanismo de retención** que unifica los beneficios de recurrencia (inferencia eficiente) con atención (entrenamiento paralelo). La innovación clave es reemplazar atención softmax con descomposición exponencial dependiente de posición, permitiendo tres paradigmas de computación: paralelo, recurrente y recurrente por bloques.

RetNet utiliza **retención multi-escala** donde cada cabeza tiene una tasa de descomposición diferente γ_h, permitiendo que diferentes cabezas capturen dependencias en diferentes escalas temporales. Logra memoria de inferencia $\mathcal{O}(1)$ y latencia, rendimiento comparable a Transformadores en modelado de lenguaje, y es compatible con entrenamiento completamente paralelizable.

A.5.2. Formulación Matemática

El mecanismo de retención en **forma paralela** es:

$$\mathbf{O} = (\mathbf{Q}\mathbf{K}^\top \odot \mathbf{D})\mathbf{V} \tag{8}$$

donde la matriz de descomposición $D_{nm} = \gamma^{n-m}$ para $n \geq m$, y 0 en otro caso. En **forma recurrente**:

$$\mathbf{S}_t = \gamma \mathbf{S}_{t-1} + \mathbf{v}_t \mathbf{k}_t^\top, \quad \mathbf{o}_t = \mathbf{S}_t \mathbf{q}_t \tag{9}$$

Para retención multi-escala, cada cabeza h usa un γ_h diferente, e información posicional se codifica vía incrustaciones complejas exponenciales estilo xPos. La **forma recurrente por bloques** procesa bloques en paralelo mientras propaga estados recurrentemente entre bloques.

A.5.3. Pseudocódigo

Algorithm 4 RetNet - Retención Multi-Escala

Require: Entrada $x_{1:L}$, tasas de descomposición $\gamma_{1:H}$ (por cabeza)
Ensure: Salida $\mathbf{O}$

```
S ← 0_{H×d×d}, O ← []
for t = 1 to L do
    q_t ← W_q(x_t), k_t ← W_k(x_t), v_t ← W_v(x_t)
        ▷ Re-parametriza con incrustaciones complejas xPos
    for h = 1 to H do
        S_h ← γ_h S_h + v_{t,h} k_{t,h}^⊤
        o_{t,h} ← S_h q_{t,h}
    end for
    Agregar [o_{t,1}; ...; o_{t,H}] a O
end for
```

A.5.4. Ventajas y Limitaciones

A.6. 5. HGRN2 (Red Recurrente con Compuertas Jerárquicas 2)

A.6.1. Descripción

HGRN2, por Qin et al. (2024), extiende HGRN introduciendo un mecanismo de **expansión de estado basado en producto exterior** que engranda significativamente el tamaño

Ventajas	Limitaciones
Tres paradigmas de computación (paralelo/recurrente/por bloques)	Tasa de descomposición fija (no dependiente de datos) limita adaptatividad
$\mathcal{O}(1)$ memoria de inferencia; sin caché KV necesario	Desempeño inferior a modelos con compuerta dependiente de datos
Sin softmax $\rightarrow$ computación más simple	Expresividad limitada para tareas de recuperación
Compatible con entrenamiento paralelo	Esquema de incrustación posicional complejo

del estado recurrente sin agregar parámetros. Mientras HGRN usa un estado oculto valuado en vectores con compuertas de olvido jerárquicas (tramos inferiores aumentan para capas superiores), HGRN2 adopta un estado valuado en matrices vía productos exteriores clave-valor, proporcionando una **interpretación de atención lineal** que permite entrenamiento eficiente en hardware.

HGRN2 en escala 3B ligeramente supera Mamba y Transformadores arquitectura-LLaMA en modelado de lenguaje.

A.6.2. Formulación Matemática

La actualización de estado HGRN2 es:

$$\mathbf{S}_t = \mathrm{diag}(\mathbf{g}_t) \cdot \mathbf{S}_{t-1} + \mathbf{v}_t \mathbf{k}_t^\top, \quad \mathbf{o}_t = \mathbf{S}_t \mathbf{q}_t \tag{10}$$

donde $\mathbf{g}_t \in \mathbb{R}^d$ es una compuerta de olvido diagonal con **valores con tramo inferior jerárquico**: el tramo inferior aumenta monótonamente en capas superiores, asegurando que capas superiores capturen dependencias de largo alcance mientras que capas inferiores modelan patrones locales. La compuerta de olvido se calcula como:

$$\mathbf{g}_t = b + (1 - b) \cdot \sigma(\mathbf{W}_g \mathbf{x}_t) \tag{11}$$

donde b es el tramo inferior dependiente de la capa.

A.6.3. Pseudocódigo

Algorithm 5 HGRN2 - Retención de Red Recurrente Jerárquicamente Compuertada

Require: Entrada $x_{1:L}$, tramo inferior por capa b
Ensure: Salida $\mathbf{O}$

```
S ← 0_{d×d}, O ← []
for t = 1 to L do
    q_t, k_t, v_t ← proyecta x_t
    g_t ← b + (1 − b) · σ(W_g(x_t))    ▷ compuerta con tramo inferior
    S ← diag(g_t) · S + v_t k_t^⊤
    o_t ← S q_t
    Agregar o_t a O
end for
```

Ventajas	Limitaciones
Expansión de estado eficiente vía productos exteriores (sin params extra)	Compuerta valuada en vectores menos flexible que regla delta
Tramos jerárquicos imponen modelado temporal multi-escala	Menos efectivo en tareas de recuperación que variantes DeltaNet
Interpretación atención lineal → entrenamiento por bloques eficiente	Brecha de desempeño vs. Transformadores persiste en contexto largo
Competitivo con Mamba en escala 3B	Requiere ajuste cuidadoso de tramos dependientes de capas

A.6.4. Ventajas y Limitaciones

A.7. 6. Transformador de Olvido (FoX)

A.7.1. Descripción

El Transformador de Olvido (FoX, Lin et al., 2025), toma un enfoque diferente de los modelos recurrentes lineales arriba: incrusta una **compuerta de olvido directamente en atención softmax** agregando un sesgo de logit dependiente de datos a las puntuaciones de atención. Esto preserva la expresividad completa y computación de tiempo cuadrático de atención softmax mientras gana beneficios de control de recencia.

FoX supera Transformadores en modelado de lenguaje de contexto largo, extrapolación de longitud y tareas de corto contexto, mientras mantiene recuperación de aguja-en-pajar cercana-perfecta. Es compatible con FlashAttention y no requiere incrustaciones posicionales.

A.7.2. Formulación Matemática

En cada paso temporal, se calcula una compuerta de olvido escalar:

$$f_t = \sigma(\mathbf{w}_f^\top \mathbf{x}_t + b_f) \tag{12}$$

La salida de **Atención de Olvido** es:

$$\mathbf{o}_i = \frac{\sum_{j=1}^{i} \exp(\mathbf{q}_i^\top \mathbf{k}_j + d_{ij})\mathbf{v}_j}{\sum_{j=1}^{i} \exp(\mathbf{q}_i^\top \mathbf{k}_j + d_{ij})} \tag{13}$$

donde $d_{ij} = \sum_{l=j+1}^{i} \log f_l$. En forma matricial:

$$\mathbf{O} = \mathrm{softmax}(\mathbf{Q}\mathbf{K}^\top + \mathbf{D})\mathbf{V}, \quad \text{donde } \mathbf{D} = \log \mathbf{F} \tag{14}$$

Esto es equivalente a una **versión aprendible y dependiente de datos** de ALiBi. La implementación agrega sumas logarítmicas acumulativas de compuertas de olvido a la computación de logit de atención de FlashAttention, con sobrecarga negligible.

A.7.3. Pseudocódigo

A.7.4. Ventajas y Limitaciones

A.8. 7. Atención con Compuerta (Sigmoide después de SDPA)

A.8.1. Descripción

El Mejor Paper NeurIPS 2025 del equipo Alibaba Qwen investiga sistemáticamente atención softmax aumentada con compuertas a través de más de 30 variantes de modelos MoE 15B y densos 1.7B entrenados en 3.5T tokens. Su hallazgo central es que aplicar una **compuerta**

Algorithm 6 FoX - Transformador de Olvido con Sesgo de Logit

Require: Consulta Q, Clave K, Valor V, parámetros compuerta w_f, b_f
Ensure: Atención con olvido $\mathbf{O}$

```
n ← longitud de secuencia
Inicializa matriz sesgo D de tamaño n × n a cero
for i = 1 to n do
    for j = 1 to i − 1 do        ▷ causal: solo mirar hacia atrás
        f_j ← σ(w_f^⊤ x_j + b_f)
        D_ij ← D_ij + log f_j    ▷ suma acumulativa de log-compuertas
    end for
end for
logits ← QK^⊤/√d_k + D           ▷ agrega sesgo de olvido
pesos ← softmax(logits)
O ← pesos · V
```

Ventajas	**Limitaciones**
Expresividad completa de softmax retenida	Aún $\mathcal{O}(L^2)$ tiempo y memoria (cuadrático)
Compatible con FlashAttention (sobrecarga minimal)	Compuerta de olvido agrega sesgo de recencia que puede perjudicar recuperación global
Sin incrustaciones posicionales necesarias	Ablaciones de tamaño de modelo solo hasta 760M parámetros
Extrapolación de longitud superior vs. Transformador	No tan eficiente en memoria en inferencia como modelos lineales

sigmoide específica de cabeza después de SDPA mejora consistentemente rendimiento, estabilidad de entrenamiento y propiedades de escalado.

Esta modificación simple introduce **no-linealidad** (rompiendo cuello de botella de rango bajo entre W_V y W_O) y **sparsidad dependiente de consulta** que elimina el fenómeno de sumidero de atención. Ya integrado en Qwen3-Next.

A.8.2. Formulación Matemática

El mecanismo de compuerta se formaliza como:

$$\mathbf{Y}' = \mathbf{Y} \odot \sigma(\mathbf{X}\mathbf{W}_\theta) \tag{15}$$

donde $\mathbf{Y}$ es la salida de SDPA y σ es la función sigmoide. Con la canalización de atención completa:

$$\mathbf{O} = \left[\text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}\right] \odot \sigma(\mathbf{X}\mathbf{W}_g) \tag{16}$$

La forma de puntuación de compuerta es $\mathbb{R}^{n\times q\times d_k}$ para compuerta elemento por elemento (donde q es número de cabezas de consulta), o $\mathbb{R}^{n\times q}$ para compuerta específica de cabeza (solo 1.6M parámetros extra para modelo MoE 15B).

Perspectiva clave: compuertas efectivas son **sparsas** (puntuación media $\approx 0{,}116$), actuando como filtros dependientes de consulta.

Algorithm 7 Atención con Compuerta - Compuerta Sigmoide post-SDPA

Require: Consulta Q, Clave K, Valor V, entrada X, parámetros compuerta W_g

Ensure: Atención compuertada $\mathbf{O}$

1: $n \leftarrow$ longitud de secuencia, $h \leftarrow$ número de cabezas
2: scores $\leftarrow (QK^\top)/\sqrt{d_k}$ ▷ puntuaciones de compatibilidad
3: pesos $\leftarrow$ softmax(scores) ▷ normalización estándar
4: $Y \leftarrow$ pesos $\cdot V$ ▷ salida de atención producto-escala
5: $g \leftarrow \sigma(XW_g)$ ▷ compuerta sigmoide (por cabeza o elemento)
6: $\mathbf{O} \leftarrow Y \odot g$ ▷ modulación por compuerta

Ventajas	**Limitaciones**
Modificación extremadamente simple (< 2 % sobrecarga de latencia)	Aún $\mathcal{O}(L^2)$ complejidad cuadrática
Elimina fenómeno de sumidero de atención	Requiere atención softmax (no aplica a variantes lineales)
Mejora estabilidad de entrenamiento; permite tasas aprendizaje mayores	Beneficio marginal para tareas de contexto corto
Variante específica de cabeza solo agrega ∼ 1,6M params para modelo 15B	Efecto parcialmente se superpone con trucos de normalización existentes

A.8.3. Pseudocódigo

A.8.4. Ventajas y Limitaciones

A.9. Tabla Comparativa Resumen

Arquitectura	**Año**	**Complejidad**	**Memoria Inf.**	**Estado Matriz**	**Compuerta Datos**
GLA	2023	$\mathcal{O}(Ld^2)$	$\mathcal{O}(d^2)$	Sí	Diagonal
DeltaNet	2024	$\mathcal{O}(Ld^2)$	$\mathcal{O}(d^2)$	Sí	Escalar β
Gated DeltaNet	2024	$\mathcal{O}(Ld^2)$	$\mathcal{O}(d^2)$	Sí	Escalar $\alpha + \beta$
RetNet	2023	$\mathcal{O}(Ld^2)$	$\mathcal{O}(d^2)$	Sí	Fija (no datos)
HGRN2	2024	$\mathcal{O}(Ld^2)$	$\mathcal{O}(d^2)$	Sí	Diagonal jer.
FoX	2025	$\mathcal{O}(L^2d)$	$\mathcal{O}(Ld)$	No	Escalar/logit
Gated Attn	2025	$\mathcal{O}(L^2d)$	$\mathcal{O}(Ld)$	No	Elem./cabeza

A.10. Taxonomía y Evolución de Arquitecturas de Compuerta

La progresión de bloques de atención con compuerta puede entenderse a través de tres "generaciones" de reglas de actualización de estado:

A.10.1. Generación 1 — Compuertas Fijas/Simples

RetNet (tasa de descomposición fija por cabeza), RWKV (no dependiente de datos). Adaptatividad limitada pero simple de implementar.

A.10.2. Generación 2 — Compuertas Dependientes de Datos

GLA, Mamba2, HGRN2 (compuertas diagonales dependientes de datos). Mejor gestión de memoria pero actualización meramente aditiva.

A.10.3. Generación 3 — Regla Delta + Compuertas

DeltaNet, Gated DeltaNet (borrar-escribir dirigido con compuertas opcionales). Mejor recuperación y gestión de memoria pero transiciones más complejas.

Ortogonalmente, **FoX** y **Gated Attention** representan dirección paralela: en lugar de reemplazar softmax con recurrencia lineal, mejoran atención softmax con mecanismos de compuerta, preservando su expresividad cuadrática mientras ganan beneficios de olvido/sparsidad.

El campo converge hacia **arquitecturas híbridas** que intercalan capas recurrentes lineales (ej., Gated DeltaNet) con capas de atención softmax dispersa, logrando lo mejor de ambos mundos. Arquitecturas como Gated DeltaNet-H1 (Gated DeltaNet + atención de ventana deslizante) superen consistentemente tanto modelos recurrentes puros como modelos de atención pura.

A. Algoritmos Avanzados de Optimización en Arquitecturas Transformadores

A.1. Introducción: Evolución de la Optimización en Redes Neuronales

La optimización de redes neuronales profundas, particularmente arquitecturas Transformadores altamente parametrizadas, representa uno de los desafíos matemáticos más complejos en la teoría del aprendizaje computacional moderno. Los paisajes de pérdida de modelos grandes de lenguaje se caracterizan por una no-convexidad severa, puntos de silla y heterogeneidad de bloques. La heterogeneidad de bloques implica que el espectro del Hessiano varía dramáticamente entre bloques de parámetros distintos, creando caminos de optimización donde ciertas dimensiones son exponencialmente más agudas o más planas que otras.

La transición de Redes Neuronales Convolucionales (CNNs) a Transformadores requirió un cambio fundamental en los paradigmas de optimización. Mientras que el Descenso de Gradiente Estocástico con Momentum dominaba tareas de visión, falló consistentemente en navegar la curvatura heterogénea de mecanismos de atención de manera eficiente. Algoritmos adaptativos, primariamente Adam y su variante desacoplada AdamW, emergieron como el estándar dominante adaptando dinámicamente la tasa de aprendizaje elemento a elemento basada en los primeros y segundos momentos de los gradientes. A medida que el conteo de parámetros de modelos escaló, la sobrecarga de memoria de mantener tensores densos de momentum y varianza impulsó el campo hacia nuevos paradigmas de optimización.

Los avances recientes han fraccionado el paisaje de optimización en varias trayectorias teóricas distintas: aproximaciones de bajo rango eficientes en memoria (Adafactor, GaLore), aproximaciones de segundo orden y matriz (Sophia, Shampoo, SOAP), estimadores de distancia libres de tasa de aprendizaje (Prodigy, Schedule-Free), técnicas de reducción de varianza (MARS, Adan), y ortogonalizadores geométricos orientados a matriz (Muon, Turbo-Muon).

A.2. Optimizadores de Línea Base y Adaptativos

A.2.1. SGD con Momentum

Descripción: El algoritmo original de referencia. Actualiza parámetros usando el gradiente escalado por una tasa de aprendizaje fija, más un término de momentum (media móvil exponencial de gradientes pasados) para acelerar convergencia y amortiguar oscilaciones.

Ventajas: Memoria mínima (solo 1 buffer para momentum), excelente generalización en muchos contextos, teoría bien entendida.

Desventajas: Requiere ajuste muy cuidadoso de cronograma de LR, convergencia lenta en tareas NLP/transformador, no adaptivo — misma LR para todos los parámetros.

Formulación Matemática: Dado la función objetivo $f_t(\theta)$ y gradiente $g_t = \nabla_\theta f_t(\theta_t)$:

$$m_t = \beta m_{t-1} + g_t \tag{17}$$

$$\theta_{t+1} = \theta_t - \eta_t m_t \tag{18}$$

Pseudocódigo:

Algorithm 8 SGD con Momentum

Require: Parámetros iniciales θ_0, tasa aprendizaje η, momentum β, peso decay λ

```
m_0 ← 0
for t = 1, 2, ..., T do
    Computar gradiente: g_t ← ∇_θ f_t(θ_{t-1})
    Aplicar decaimiento: g_t ← g_t + λθ_{t-1}
    Actualizar momentum: m_t ← βm_{t-1} + g_t
    Actualizar parámetros: θ_t ← θ_{t-1} − ηm_t
end for
```

A.2.2. Adam y AdamW

Descripción: Combina momentos de primer orden (media) y segundo orden (varianza no centrada) de medias móviles exponenciales de gradientes. Usa corrección de sesgo para compensar la inicialización en cero de estas estimaciones. AdamW desacopla fundamentalmente el decaimiento de pesos del mecanismo de actualización de gradientes, asegurando que la regularización penalice los pesos directamente.

Ventajas: Convergencia rápida, robusto a hiperparámetros, tasa de aprendizaje adaptiva por parámetro, estándar de facto para ajuste fino.

Desventajas: Problemas de no-convergencia teórica con ciertos β_2, puede converger a mínimos agudos con peor generalización, requiere memoria $2\times$ de optimizer states.

Formulación Matemática:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t \tag{19}$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2 \tag{20}$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t} \tag{21}$$

$$\theta_{t+1} = \theta_t - \eta_t \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda \theta_t \right) \tag{22}$$

Pseudocódigo (AdamW):

Algorithm 9 AdamW — Adam Desacoplado

Require: Parámetros θ_0, tasa aprendizaje η, betas (β_1, β_2), epsilon ϵ, peso decay λ

```
m_0, v_0 ← 0
for t = 1, 2, ..., T do
    Computar gradiente: g_t ← ∇_θ f_t(θ_{t-1})
    Decaimiento de pesos desacoplado: θ_{t-1} ← (1 − λη)θ_{t-1}
    Actualizar momentos: m_t ← β_1 m_{t-1} + (1 − β_1) g_t
    Actualizar segundos momentos: v_t ← β_2 v_{t-1} + (1 − β_2) g_t^2
    Corregir sesgo: m̂_t ← m_t/(1−β_1^t), v̂_t ← v_t/(1−β_2^t)
    Actualizar parámetros: θ_t ← θ_{t-1} − η m̂_t/(√v̂_t + ε)
end for
```

A.2.3. RAdam (Rectificado Adam)

Descripción: Identifica que la tasa de aprendizaje adaptiva de Adam tiene una varianza problemáticamente grande en etapas tempranas de entrenamiento (explicando por qué se necesita calentamiento). RAdam introduce un término de rectificación de varianza automático, cambiando dinámicamente entre comportamiento SGD y Adam dependiendo de la varianza estimada.

Ventajas: Elimina la necesidad de ajuste manual de calentamiento, más robusto sin ajuste cuidadoso, reemplazo directo para Adam.

Desventajas: Ganancias marginales sobre Adam bien ajustado + calentamiento, no ampliamente adoptado en preentrenamiento a gran escala.

Formulación Matemática: Calcular la longitud máxima de la SMA aproximada: $\rho_\infty = \frac{2}{1-\beta_2} - 1$. En paso t, calcular grados de libertad: $\rho_t = \rho_\infty - \frac{2t\beta_2^t}{1-\beta_2^t}$. Si $\rho_t > 4$, computar término de rectificación de varianza r_t y actualizar adaptativamente:

$$r_t = \sqrt{\frac{(\rho_t - 4)(\rho_t - 2)\rho_\infty}{(\rho_\infty - 4)(\rho_\infty - 2)\rho_t}} \tag{23}$$

$$\theta_{t+1} = \theta_t - \eta_t r_t \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \tag{24}$$

Si $\rho_t \leq 4$, comportarse como SGD: $\theta_{t+1} = \theta_t - \eta_t \hat{m}_t$.

A.3. Momentum Avanzado y Reducción de Varianza (2024-2025)

A.3.1. Adan (Momentum Nesterov Adaptivo)

Descripción: Reformula la aceleración de Nesterov para evitar el cálculo extra de gradientes en el punto de extrapolación. Usa Estimación de Momentum Nesterov (NME) para ambas estimaciones de momentos primer y segundo orden. Funciona bien en arquitecturas diversas (CNNs, Transformadores, GANs).

Ventajas: Convergencia consistentemente más rápida en arquitecturas, evita paso forward/backward extra de Nesterov, fuerte en tareas visión y lenguaje.

Desventajas: 3 buffers de momentum → mayor memoria que Adam, relativamente nuevo, puede ser inestable cerca de convergencia.

Formulación Matemática:

$$m_k = (1 - \beta_1)m_{k-1} + \beta_1 g_k \tag{25}$$

$$v_k = (1 - \beta_2)v_{k-1} + \beta_2(g_k - g_{k-1}) \tag{26}$$

$$n_k = (1 - \beta_3)n_{k-1} + \beta_3[g_k + (1 - \beta_1)(g_k - g_{k-1})]^2 \tag{27}$$

$$\theta_{k+1} = \theta_k - \eta \frac{m_k + (1 - \beta_1)v_k}{\sqrt{n_k} + \epsilon} \tag{28}$$

Pseudocódigo:

A.3.2. ADOPT (Adam Modificado con Convergencia Óptima)

Descripción: Corrige la no-convergencia teórica de Adam por: (1) removiendo el gradiente actual de la estimación de segundo momento, y (2) reordenando la actualización de momentum y normalización. Alcanza la tasa óptima $\mathcal{O}(1/\sqrt{T})$ con cualquier elección de β_2, sin suposiciones de ruido acotado.

Ventajas: Probadamente converge con cualquier β_2, reemplazo directo para Adam, superior en visión, NLP, RL, tareas generativas.

Algorithm 10 Adan — Momentum Nesterov Adaptivo

Require: Parámetros θ_0, tasa aprendizaje η, betas $(\beta_1, \beta_2, \beta_3)$

$m_0, v_0, n_0, g_{-1} \leftarrow 0$
for $k = 1, 2, \ldots, T$ **do**
  Computar gradiente: $g_k \leftarrow \nabla_\theta f(\theta_{k-1})$
  Diferencia de gradientes: $\Delta g_k \leftarrow g_k - g_{k-1}$
  Actualizar primer momento: $m_k \leftarrow (1-\beta_1)m_{k-1} + \beta_1 g_k$
  Actualizar segundo momento: $v_k \leftarrow (1-\beta_2)v_{k-1} + \beta_2 \Delta g_k$
  Nesterov proyectado: $g_{nesterov} \leftarrow g_k + (1-\beta_1)\Delta g_k$
  Actualizar momento cuadrado: $n_k \leftarrow (1-\beta_3)n_{k-1} + \beta_3 g_{nesterov}^2$
  Actualizar parámetros: $\theta_k \leftarrow \theta_{k-1} - \eta \frac{m_k + (1-\beta_1)v_k}{\sqrt{n_k}+\epsilon}$
end for

Desventajas: Ganancias prácticas marginales en algunas tareas, muy reciente (NeurIPS 2024).

Formulación Matemática:

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2 \tag{29}$$

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) \frac{g_t}{\sqrt{v_{t-1}} + \epsilon} \tag{30}$$

$$\theta_{t+1} = \theta_t - \eta_t m_t \tag{31}$$

A.3.3. AdEMAMix (Mezcla de Dos EMAs)

Descripción: Reemplaza la EMA única de gradientes de Adam con una mezcla de dos EMAs — una de descomposición rápida (gradientes recientes) y una de descomposición lenta (memoria larga). Demuestra que los gradientes permanecen relevantes por decenas de miles de pasos.

Ventajas: Modelo 1.3B en 101B tokens AdamW en 197B tokens (+95

Desventajas: Introduce hiperparámetros extra (dos tasas descomposición + mezcla), necesita cronograma para peso EMA lento, relativamente nuevo.

Formulación Matemática:

$$m_{1,t} = \beta_1 m_{1,t-1} + (1-\beta_1) g_t \tag{32}$$

$$m_{2,t} = \beta_3 m_{2,t-1} + (1-\beta_3) g_t \tag{33}$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2 \tag{34}$$

$$m_t = m_{1,t} + \alpha m_{2,t} \tag{35}$$

$$\theta_{t+1} = \theta_t - \eta_t \frac{m_t}{\sqrt{v_t} + \epsilon} \tag{36}$$

Pseudocódigo:

A.3.4. MARS (Make Variance Reduction Shine)

Descripción: Marco unificado que reconcilia métodos de gradiente precondicionados con reducción de varianza vía momentum recursivo estocástico escalado. Ofrece instancias basadas en AdamW, Lion, y Shampoo.

Ventajas: Trae reducción de varianza a entrenamiento de modelos grandes (una primicia), supera consistentemente AdamW por margen considerable.

Desventajas: Memoria extra para estimaciones de gradientes con varianza reducida, implementación más compleja.

Algorithm 11 AdEMAMix — Mezcla de Medias Móviles Exponenciales

Require: Parámetros θ_0, betas $(\beta_1, \beta_2, \beta_3)$, coeficiente mezcla α

$m_{1,0}, m_{2,0}, v_0 \leftarrow 0$
for $t = 1, 2, \ldots, T$ **do**
  Computar gradiente: $g_t \leftarrow \nabla_\theta f(\theta_{t-1})$
  EMA rápida: $m_{1,t} \leftarrow \beta_1 m_{1,t-1} + (1 - \beta_1) g_t$
  EMA lenta: $m_{2,t} \leftarrow \beta_3 m_{2,t-1} + (1 - \beta_3) g_t$
  Segundo momento: $v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$
  Mezcla momento: $m_t \leftarrow m_{1,t} + \alpha m_{2,t}$
  Actualizar: $\theta_t \leftarrow \theta_{t-1} - \eta \frac{m_t}{\sqrt{v_t} + \epsilon}$
end for

Formulación Matemática (variante MARS-AdamW):

$$c_t = g_t + \gamma_t (c_{t-1} - g_{t-1}) \tag{37}$$
$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) c_t \tag{38}$$
$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) c_t^2 \tag{39}$$
$$\theta_{t+1} = \theta_t - \eta_t \frac{m_t}{\sqrt{v_t} + \epsilon} \tag{40}$$

Pseudocódigo:

Algorithm 12 MARS — Marco de Reducción de Varianza

Require: Parámetros θ_0, coeficiente reducción varianza γ

$m_0, v_0, c_0, g_{-1} \leftarrow 0$
for $t = 1, 2, \ldots, T$ **do**
  Computar gradiente: $g_t \leftarrow \nabla_\theta f(\theta_{t-1})$
  Momentum recursivo: $c_t \leftarrow g_t + \gamma_t (c_{t-1} - g_{t-1})$
  Actualizar momentos con c_t (no g_t):
    $m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) c_t$
    $v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2) c_t^2$
  Actualizar parámetros: $\theta_t \leftarrow \theta_{t-1} - \eta \frac{m_t}{\sqrt{v_t} + \epsilon}$
end for

A.3.5. Optimizadores Cautelosos (C-AdamW, C-Lion)

Descripción: Modificación de una sola línea a cualquier optimizador basado en momentum: la actualización se enmascara de forma que solo componentes donde el momentum y gradiente acuerdan en signo se aplican. Preserva garantías de convergencia mientras acelera significativamente el entrenamiento.

Ventajas: Hasta 1.47× aceleración en preentrenamiento Llama/MAE, trivial de implementar (una línea), funciona sobre AdamW, Lion, o cualquier método momentum.

Desventajas: Efecto marginal en algunas tareas, enmascaramiento puede ralentizar entrenamiento temprano, muy reciente.

Formulación Matemática:

$$\text{máscara}_t = \mathbb{I}(m_t \odot g_t > 0) \tag{41}$$
$$\theta_{t+1} = \theta_t - \eta_t (\text{ActualizaciónBase}_t \odot \text{máscara}_t) \tag{42}$$

A.4. Optimizadores Eficientes en Memoria y a Gran Escala

A.4.1. LAMB (Aprendizaje Adaptativo Basado en Capas)

Descripción: La necesidad de entrenar LLMs en datasets masivos ha impulsado la adopción de tamaños de lote extremos. LAMB resuelve degradación de generalización combinando un tracker de momentum estilo Adam con una relación de confianza por capa derivada de Escalado de Tasa Adaptativa por Capa (LARS).

Ventajas: Entrena eficientemente con lotes extremadamente grandes, convergencia rápida en modelos BERT.

Desventajas: Implementación más compleja que Adam, comportamiento menos predecible con ciertos cronogramas.

Formulación Matemática:

$$\phi(\|x_t^{(i)}\|) = \frac{\|x_t^{(i)}\|}{\|r_t^{(i)}\|} \tag{43}$$

$$x_{t+1}^{(i)} = x_t^{(i)} - \eta_t \phi(\|x_t^{(i)}\|) \frac{r_t^{(i)}}{\|r_t^{(i)}\|} \tag{44}$$

A.4.2. Schedule-Free (AdamW sin Cronograma)

Descripción: Remueve completamente la necesidad de cronograma de tasa de aprendizaje unificando cronograma e iteración promediada en un marco teórico único. No introduce hiperparámetros extra.

Ventajas: No necesita especificar pasos totales de entrenamiento T, estado del arte en convexo a aprendizaje profundo a gran escala, sin hiperparámetros extra.

Desventajas: Requiere cambiar entre vistas de parámetros *entrenamiento* y *evaluación*, API ligeramente diferente que optimizadores estándar.

Formulación Matemática:

$$z_{t+1} = z_t - \eta \nabla f(y_t) \tag{45}$$

$$x_{t+1} = \left(1 - \frac{1}{t+1}\right) x_t + \frac{1}{t+1} z_{t+1} \tag{46}$$

$$y_{t+1} = (1-\beta) z_{t+1} + \beta x_{t+1} \tag{47}$$

A.4.3. Adafactor (Factorización Inteligente de Varianza)

Descripción: Reduce memoria de optimizer factorizando estadísticas de segundo momento para tensores tipo matriz, almacenando acumuladores por fila/columna en lugar de estados de varianza densos.

Ventajas: Memoria sublineal en dimensión parámetro, excelente para transformadores grandes, learning-rate-free.

Desventajas: Menos pulido que Adam, aún bajo investigación para casos edge.

Formulación Matemática: Dado gradiente $G_t \in \mathbb{R}^{n \times m}$:

$$R_t = \beta_2 R_{t-1} + (1-\beta_2)(G_t^2)\mathbf{1}_m \tag{48}$$

$$C_t = \beta_2 C_{t-1} + (1-\beta_2)\mathbf{1}_n^\top (G_t^2) \tag{49}$$

$$\hat{V}_t = \frac{R_t C_t / (\mathbf{1}_n^\top R_t)}{1-\beta_2^t} \tag{50}$$

A.4.4. GaLore (Proyección de Gradientes de Bajo Rango)

Descripción: Proyecta gradientes 2D en subespacio de bajo rango, optimiza en ese espacio compacto, y reconstruye actualizaciones en espacio original. Reduce memoria de optimizer a $\mathcal{O}(Nr)$ donde r es rango objetivo.

Ventajas: Entrenamiento significativamente más eficiente en memoria, funciona con arquitecturas existentes, compatible con cuantización.

Desventajas: Overhead computacional de SVD periódico, requiere selección cuidadosa de rango.

Formulación Matemática:

$$U, S, V = \mathrm{SVD}(G_t), \quad P = U_{[:,:r]} \text{ o } V_{[:,:r]} \tag{51}$$

$$G_{\text{low}} = P^\top G_t \text{ (o } G_t P^\top) \tag{52}$$

$$\Delta = P\Delta_{\text{low}} \text{ (o } \Delta_{\text{low}} P) \tag{53}$$

A.5. Métodos Libres de Parámetros y Adaptativos a Distancia

A.5.1. Prodigy (Adaptación de Distancia Dinámica)

Descripción: Prodigy adapta escala de paso efectiva mediante estadístico tipo distancia, reduciendo sensibilidad a ajuste manual de LR.

Ventajas: Eliminación de sintonización LR, comportamiento robusto sobre tamaños de lote variable.

Desventajas: Overhead computacional adicional para estadístico distancia, menos prueba exhaustiva que Adam.

Formulación Matemática:

$$s_k = \sum_t \langle u_t, p_t - p_0 \rangle \tag{54}$$

$$d_t = \text{máx}(d_{t-1}, d_0 + d_{\text{coef}} \cdot s_k) \tag{55}$$

$$\theta_{t+1} = \theta_t - \eta \cdot d_t \cdot u_t \tag{56}$$

A.6. Optimizadores de Segundo Orden y Geométricos

A.6.1. Shampoo (Precondicionadores Factorizados Kronecker)

Descripción: Optimizador de segundo orden estructurado que mantiene matrices precondicionadoras factorizadas Kronecker (una por dimensión de cada tensor parámetro). Aproxima AdaGrad de matriz completa con costo manejable.

Ventajas: Garantías de convergencia teórica fuerte, captura correlaciones entre parámetros, probado a escala de Google.

Desventajas: Memoria alta para matrices precondicionadoras, requiere eigendecomposición periódica costosa, implementación distribuida compleja.

Formulación Matemática: Para gradiente de matriz $G_t \in \mathbb{R}^{n \times m}$:

$$L_t = L_{t-1} + G_t G_t^\top, \quad R_t = R_{t-1} + G_t^\top G_t \tag{57}$$

$$W_{t+1} = W_t - \eta L_t^{-1/4} G_t R_t^{-1/4} \tag{58}$$

Pseudocódigo:

Algorithm 13 Shampoo — Preacondicionamiento Kronecker Factorizado

Require: Parámetros matriz $W_0 \in \mathbb{R}^{n \times m}$, frecuencia actualización f

Inicializar: $L_0 = \epsilon I_n$, $R_0 = \epsilon I_m$
for $t = 1, 2, \ldots, T$ **do**
  Computar gradiente: $G_t \leftarrow \nabla_W f(W_{t-1})$
  Acumular Kronecker: $L_t \leftarrow L_{t-1} + G_t G_t^\top$, $R_t \leftarrow R_{t-1} + G_t^\top G_t$
  if $t \mod f = 0$ **then**
    Computar raíces inversas:
      $L_t^{-1/4} \leftarrow Q_L \Lambda_L^{-1/4} Q_L^\top$ (eigendecomposición)
      $R_t^{-1/4} \leftarrow Q_R \Lambda_R^{-1/4} Q_R^\top$
  end if
  Actualizar parámetros: $W_t \leftarrow W_{t-1} - \eta L_t^{-1/4} G_t R_t^{-1/4}$
end for

A.6.2. SOAP (Shampoo con Adam en Base Propia)

Descripción: Versión simplificada y mejorada de Shampoo. Idea clave: ejecutar Adam/Adafactor en la base propia de precondicionadores de Shampoo produce algoritmo más simple y rápido.

Ventajas: Computacionalmente más simple que Shampoo, combina beneficios de Adam y Shampoo, resultados sólidos entrenamiento LLM.

Desventajas: Aún requiere eigendecomposición (menos frecuentemente), memoria alta que métodos primer orden.

Formulación Matemática: Eigen-descomponer precondicionadores: $L = Q_L \Lambda_L Q_L^\top$, $R = Q_R \Lambda_R Q_R^\top$. Proyectar gradiente: $G_{\text{rot}} = Q_L^\top G Q_R$. Ejecutar Adam estándar en G_{rot} para obtener $M_{\text{rot}}, V_{\text{rot}}$. Proyectar actualización de vuelta: $\Delta = Q_L \left(\frac{M_{\text{rot}}}{\sqrt{V_{\text{rot}}}} \right) Q_R^\top$.

A.6.3. Sophia (Hessiana Diagonal y Clipping)

Descripción: Usa estimaciones periódicas de Hessiana diagonal con clipping de actualizaciones escaladas por segundo orden para optimización estable a gran escala.

Ventajas: Convergencia acelerada incorporando curvatura, overhead bajo comparado a segundo orden completo.

Desventajas: Estimación Hessiana diagonal puede ser ruidosa, requiere tuning de parámetro clipping.

Formulación Matemática:

$$h_t = \beta_2 h_{t-k} + (1 - \beta_2) \hat{h}_t \tag{59}$$

$$\theta_{t+1} = \theta_t - \eta_t \cdot \text{clip} \left(\frac{m_t}{\text{máx}(\gamma h_t, \epsilon)}, 1 \right) \tag{60}$$

A.7. Optimizadores Basados en Ortogonalidad

A.7.1. Lion (EvoLved Sign Momentum)

Descripción: Lion usa actualizaciones basadas en signo con rastreo de momentum, reduciendo huella de memoria comparado a métodos tipo Adam de segundo momento.

Ventajas: 50 % menos memoria que Adam, convergencia rápida, simple de implementar.

Desventajas: Menos explorado que Adam, puede necesitar ajuste de hiperparámetros cuidadoso.

Formulación Matemática:

$$c_t = \beta_1 m_t + (1 - \beta_1) g_t \tag{61}$$
$$\theta_{t+1} = \theta_t - \eta_t(\text{sign}(c_t) + \lambda\theta_t) \tag{62}$$
$$m_{t+1} = \beta_2 m_t + (1 - \beta_2) g_t \tag{63}$$

A.7.2. Muon (Ortonormalización Newton-Schulz)

Descripción: Muon ortonormaliza actualizaciones momentum para tensores 2D usando iteraciones de Newton-Schulz. Combina eficiencia de memoria de Lion con mejor convergencia.

Ventajas: Convergencia consistentemente rápida, bajo costo computacional, excelente para transformadores grandes.

Desventajas: Solo se aplica a tensores 2D (matrices), requiere tuning de iteraciones Newton-Schulz.

Formulación Matemática:

$$X_0 = \frac{G}{\|G\|_F + \epsilon} \tag{64}$$
$$A_k = X_k X_k^\top, \quad B_k = bA_k + cA_k^2 \tag{65}$$
$$X_{k+1} = aX_k + B_k X_k \tag{66}$$

Pseudocódigo:

Algorithm 14 Muon — Momentum con Ortonormalización

Require: Matriz parámetros $M \in \mathbb{R}^{n\times m}$, iteraciones NS $k_{\text{máx}}$

```
for t = 1, 2, ..., T do
    Computar gradiente: G_t ← ∇_M f(M_{t-1})
    Normalizar: X_0 ← G_t / (||G_t||_F + ε)
    Iterar Newton-Schulz (5 pasos típicamente):
    for k = 1, ..., k_máx do
        A ← X_{k-1} X_{k-1}^⊤
        X_k ← X_{k-1} − (1/2)(A − 3I) X_{k-1}
    end for
    Actualizar parámetros: M_t ← M_{t-1} − η X_{k_máx}
end for
```

A.7.3. Turbo-Muon (Muon Acelerado con Preacondicionamiento Casi-Ortogonal)

Descripción: Turbo-Muon añade preacondicionamiento casi-ortogonal antes de actualizaciones Newton-Schulz, reduciendo número de iteraciones ortonormalización requeridas.

Ventajas: Versión acelerada de Muon, iteraciones óptimas reducidas, mantiene mismas garantías.

Desventajas: Overhead preacondicionamiento AOL, aún muy reciente (2025).

Formulación Matemática:

$$A_0 = X_0^\top X_0, \quad s_i = (\|A_{0i}\|_1)^{-1/2} \tag{67}$$
$$X_1 = X_0 \cdot \text{diag}(s), \quad A_1 = \text{diag}(s) A_0 \text{diag}(s) \tag{68}$$

Optimizador	Orden Matemático	Complejidad	Memoria	Caso Uso
SGD + Momentum	Primer orden	$\mathcal{O}(N)$	1×	Referencia clásica
AdamW	Primer orden	$\mathcal{O}(N)$	2×	Estándar de facto
RAdam	Primer orden	$\mathcal{O}(N)$	2×	Sin calentamiento
Adan	Primer orden	$\mathcal{O}(N)$	3×	Visión + NLP
ADOPT	Primer orden	$\mathcal{O}(N)$	2×	Convergencia teórica
AdEMAMix	Primer orden	$\mathcal{O}(N)$	3×	Pre-entrenamiento gran escala
MARS	Primer orden	$\mathcal{O}(N)$	3×	Reducción varianza
Schedule-Free	Primer orden	$\mathcal{O}(N)$	2×	Sin cronograma LR
Lion	Primer orden	$\mathcal{O}(N)$	1×	Eficiencia memoria
Adafactor	Primer orden	$\mathcal{O}(N)$	Sublineal	Transformadores grandes
GaLore	Primer orden	$\mathcal{O}(Nr)$	$\mathcal{O}(Nr)$	Bajo rango proyección
Prodigy	Primer orden	$\mathcal{O}(N)$	2×	Libre hiperparámetros
Shampoo	Segundo orden	$\mathcal{O}(N + d^3)$	Alto	Ortogonalizadores precisos
SOAP	Segundo orden	$\mathcal{O}(N + d^3)$	Alto	Simplificación Shampoo
Sophia	Segundo orden	$\mathcal{O}(N)$	2×	Hessiana diagonal
Muon	Ortogonal	$\mathcal{O}(N)$	1×	Transformadores LLM
Turbo-Muon	Ortogonal	$\mathcal{O}(N)$	1×	Muon acelerado

A.8. Tabla Comparativa: Síntesis de Optimizadores

A.9. Conclusión del Apéndice de Optimización

El paisaje matemático de optimización de Transformadores se fractura entre: (i) escalado independiente de dimensiones de parámetros (AdamW, Schedule-Free, ADOPT), (ii) extracción explícita de representaciones de curvatura (Shampoo, SOAP, Sophia), (iii) explotación de bonos de trayectoria de gradientes (MARS, Cautious, AdEMAMix), y (iv) tratamiento de proyecciones de pesos como variedades geométricas restringidas (Muon, Turbo-Muon). Navegar esta jerarquía compleja permite a ingenieros de aprendizaje profundo localizar la intersección exacta de overhead de memoria, throughput FLOP, y límites de convergencia personalizados a sus restricciones específicas de hardware.

Bibliografía

[1] Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to memorize at test time, . URL http://arxiv.org/abs/2501.00663.

[2] Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to memorize at test time, . URL https://arxiv.org/abs/2501.00663. Version Number: 1.

[3] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer, 2020. URL https://arxiv.org/abs/2004.05150.

[4] Thibaut Boissin, Thomas Massena, Franck Mamalet, and Mathieu Serrurier. Turbo-muon: Accelerating orthogonality-based optimization with pre-conditioning. URL https://arxiv.org/abs/2512.04632. Version Number: 1.

[5] Riccardo Bravin, Massimo Pavan, Hazem Hesham Yousef Shalby, Fabrizio Pittorino, and Manuel Roveri. EmbBERT: Attention under 2 MB memory. URL http://arxiv.org/abs/2502.10001.

[6] Xiangning Chen, Chen Liang, Da Huang, Esteban Real, Kaiyuan Wang, Yao Liu, Hieu Pham, Xuanyi Dong, Thang Luong, Cho-Jui Hsieh, Yifeng Lu, and Quoc V. Le. Symbolic discovery of optimization algorithms. URL https://arxiv.org/abs/2302.06675. Version Number: 4.

[7] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers, 2019. URL https://arxiv.org/abs/1904.10509.

[8] Aaron Defazio, Xingyu Alice Yang, Harsh Mehta, Konstantin Mishchenko, Ahmed Khaled, and Ashok Cutkosky. The road less scheduled. URL https://arxiv.org/abs/2405.15682. Version Number: 4.

[9] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. URL http://arxiv.org/abs/1810.04805.

[10] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. URL https://arxiv.org/abs/2312.00752. Version Number: 2.

[11] Albert Gu, AI21 Labs, et al. Jamba: A hybrid transformer-mamba language model, 2024. URL https://arxiv.org/abs/2403.19887.

[12] Vineet Gupta, Tomer Koren, and Yoram Singer. Shampoo: Preconditioned stochastic tensor optimization. URL https://arxiv.org/abs/1802.09568. Version Number: 2.

[13] Sukjun Hwang, Aakash Lahoti, Tri Dao, and Albert Gu. Hydra: Bidirectional state space models through generalized matrix mixers. URL http://arxiv.org/abs/2407.09941.

[14] Prannay Khosla, Piotr Teterwak, Chen Wang, Aaron Sarna, Yonglong Tian, Phillip Isola, Aaron Maschinot, Ce Liu, and Dilip Krishnan. Supervised contrastive learning, 2020. URL https://arxiv.org/abs/2004.11362.

[15] Mike Lewis, Shruti Bhosale, Tim Dettmers, Douwe Kiela, and Luke Zettlemoyer. Base layers: Simplifying training of large, sparse models, 2021. URL https://arxiv.org/abs/2103.16716.

[16] Kaizhao Liang, Lizhang Chen, Bo Liu, and Qiang Liu. Cautious optimizers: Improving training with one line of code. URL https://arxiv.org/abs/2411.16085. Version Number: 4.

[17] Zhixuan Lin, Ke Wang, et al. Forgetting transformer: Softmax attention with a forget gate, 2025. URL https://arxiv.org/abs/2503.02130.

[18] Hong Liu, Zhiyuan Li, David Hall, Percy Liang, and Tengyu Ma. Sophia: A scalable stochastic second-order optimizer for language model pre-training, . URL https://arxiv.org/abs/2305.14342. Version Number: 4.

[19] Liyuan Liu, Haoming Jiang, Pengcheng He, Weizhu Chen, Xiaodong Liu, Jianfeng Gao, and Jiawei Han. On the variance of the adaptive learning rate and beyond, . URL https://arxiv.org/abs/1908.03265. Version Number: 4.

[20] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. URL https://arxiv.org/abs/1711.05101. Version Number: 3.

[21] Tianyu Lou, Zheyu Chen, Tao Yu, et al. Efficient sparse attention for long-range transformers, 2024. URL https://arxiv.org/abs/2406.16747.

[22] Konstantin Mishchenko and Aaron Defazio. Prodigy: An expeditiously adaptive parameter-free learner. URL https://arxiv.org/abs/2306.06101. Version Number: 4.

[23] Niklas Muennighoff et al. Matryoshka representation learning, 2022. URL https://arxiv.org/abs/2205.13147.

[24] Matteo Pagliardini, Pierre Ablin, and David Grangier. The AdEMAMix optimizer: Better, faster, older. URL https://arxiv.org/abs/2409.03137. Version Number: 2.

[25] Zhen Qin, Xu Han, et al. Hgrn2: Gated linear rnns with state expansion, 2024. URL https://arxiv.org/abs/2404.07904.

[26] Yuxiang Qiu, Qwen Team, et al. Gated attention for large language models, 2025. URL https://arxiv.org/abs/2505.06708.

[27] Jason Ramapuram, Federico Danieli, Eeshan Dhekane, Floris Weers, Dan Busbridge, Pierre Ablin, Tatiana Likhomanenko, Jagrit Digani, Zijin Gu, Amitis Shidani, and Russ Webb. Theory, analysis, and best practices for sigmoid self-attention. URL https://arxiv.org/abs/2409.04431. Version Number: 2.

[28] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. URL http://arxiv.org/abs/1908.10084.

[29] Hema Hariharan Samson. Lightweight transformer architectures for edge devices in real-time applications. URL http://arxiv.org/abs/2601.03290.

[30] Noam Shazeer and Mitchell Stern. Adafactor: Adaptive learning rates with sublinear memory cost. URL https://arxiv.org/abs/1804.04235. Version Number: 1.

[31] Wei Shen, Ruichuan Huang, Minhui Huang, Cong Shen, and Jiawei Zhang. On the convergence analysis of muon. URL https://arxiv.org/abs/2505.23737. Version Number: 1.

[32] Rafael Soares et al. Griffin: Mixing gated linear recurrences with local attention for efficient sequence modeling, 2024. URL https://arxiv.org/abs/2402.19427.

[33] Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, and Furu Wei. Retentive network: A successor to transformer for large language models. URL https://arxiv.org/abs/2307.08621. Version Number: 4.

[34] Shohei Taniguchi, Keno Harada, Gouki Minegishi, Yuta Oshima, Seong Cheol Jeong, Go Nagahara, Tomoshi Iiyama, Masahiro Suzuki, Yusuke Iwasawa, and Yutaka Matsuo. ADOPT: Modified adam can converge with any $_2$ with the optimal rate. URL https://arxiv.org/abs/2411.02853. Version Number: 3.

[35] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. URL https://arxiv.org/abs/1706.03762. Version Number: 7.

[36] Nikhil Vyas, Depen Morwani, Rosie Zhao, Mujin Kwun, Itai Shapira, David Brandfonbrener, Lucas Janson, and Sham Kakade. SOAP: Improving and stabilizing shampoo using adam. URL https://arxiv.org/abs/2409.11321. Version Number: 2.

[37] Hongyu Wang, Shuming Ma, Li Dong, Shaohan Huang, Huaijie Wang, Lingxiao Ma, Fan Yang, Ruiping Wang, Yi Wu, and Furu Wei. BitNet: Scaling 1-bit transformers for large language models. URL http://arxiv.org/abs/2310.11453.

[38] Zhe Wang, Ming Liu, et al. Fasa: Frequency-aware sparse attention, 2026. URL https://arxiv.org/abs/2602.03152.

[39] Xingyu Xie, Pan Zhou, Huan Li, Zhouchen Lin, and Shuicheng Yan. Adan: Adaptive nesterov momentum algorithm for faster optimizing deep models. URL https://arxiv.org/abs/2208.06677. Version Number: 5.

[40] Haiqi Yang, Zhiyuan Li, Yi Chang, and Yuan Wu. A survey of retentive network. URL http://arxiv.org/abs/2506.06708.

[41] Songlin Yang, Bailin Wang, et al. Gated linear attention transformers with hardware-efficient training, 2023. URL https://arxiv.org/abs/2312.06635.

[42] Songlin Yang, Bailin Wang, et al. Parallelizing linear transformers with the delta rule over sequence length, 2024. URL https://arxiv.org/abs/2406.06484.

[43] Songlin Yang, Bailin Wang, et al. Gated delta networks: Improving mamba2 with delta rule, 2024. URL https://arxiv.org/abs/2412.06464.

[44] Han Yuan, DeepSeek-AI, et al. Native sparse attention: Hardware-aligned and natively trainable sparse attention, 2025. URL https://arxiv.org/abs/2502.11089.

[45] Huizhuo Yuan, Yifeng Liu, Shuang Wu, Xun Zhou, and Quanquan Gu. MARS: Unleashing the power of variance reduction for training large models. URL https://arxiv.org/abs/2411.10438. Version Number: 4.

[46] Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Pike Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big bird: Transformers for longer sequences. In *Advances in Neural Information Processing Systems*, 2020. doi: 10.48550/ARXIV.2007.14062. URL https://arxiv.org/abs/2007.14062.

[47] Jing Zhang, Peng Zhang, Baiwen Kong, Junqiu Wei, and Xin Jiang. Continuous self-attention models with neural ODE networks. 35(16):14393–14401. ISSN 2374-3468, 2159-5399. doi: 10.1609/aaai.v35i16.17692. URL https://ojs.aaai.org/index.php/AAAI/article/view/17692.

[48] Yichi Zhang, Yizhong Wang, et al. Accurate and training-free sparse attention accelerating any model inference, 2025. URL https://arxiv.org/abs/2502.18137.

[49] Jiawei Zhao, Zhenyu Zhang, Beidi Chen, Zhangyang Wang, Anima Anandkumar, and Yuandong Tian. GaLore: Memory-efficient LLM training by gradient low-rank projection. URL https://arxiv.org/abs/2403.03507. Version Number: 2.